

Conversions from ranges to containers

Document #:	P1206R7
Date:	2022-01-21
Programming Language C++	
Audience:	LWG
Reply-to:	Corentin Jabot < corentin.jabot@gmail.com > Eric Niebler < eric.niebler@gmail.com > Casey Carter < casey@carter.net >

Abstract

We propose facilities to make constructing containers from ranges more convenient.

Revisions

Revision 7

Wording changes

- Rename the variable `range` to `rg`
- Fix a number of typos and presentation issues
- Fix the table of requirements to require *Cpp17EmplaceConstructible* where appropriate, use `*ranges::begin(rg)` instead of `range_reference_t<T>` as we are referring to a value rather than a type
- Clarify vector constructor complexity
- `ranges::from_iterator_t` was incorrectly spelled in a number of places
- Fix incorrect `ranges::` and `std::` qualifications
- Fix added deduction guides to take ranges by forwarding ref (which is needed to protect against array decay)
- Rename *range-compatible-with-container-range-construction* to *container-compatible-range*
- Use *container-compatible-range* in the description of requirement tables.
- Constrain `ranges::to` and `container::from_range_t` overload with `convertible_to` consistently. The concern was that if `ranges::to` is not properly constrained, it might try to construct improperly constrained non-standard types.
- Remove unnecessary constraints from `basic_string::assign_range`
- Add missing `constexpr` in a few places in vector

- Add missing `std::` forward in a few places
- Remove unnecessary constraints on `basic_string::append_range/replace_with_range`
- simplify the reservable-container concept specification
- make `ranges::to` concepts into `constexpr` variables to make it clearer no semantic constraints is expected
- simplify and fix `ranges::to` wording and description
- introduce `__cpp_lib_ranges_to_container` instead of bumping `__cpp_lib_ranges`

Revision 6

- At LEWG's request, rename `push_front_range` and `push_back_range` to `prepend_range` and `append_range` respectively. (`push_range` in containers adaptors is not renamed!)

Revision 5

- Add `push_back_range` and `push_front_range` methods at SG9 request
- Fix some wording issues
- Add notes on performance + benchmarks

Revision 4

- Add `from_range_t` and methods taking ranges to most containers
- Improve the wording of `ranges::to`
- `ranges::to` calls `reserve` when possible
- Rewrite the motivation

Revision 3

- Add support for `from_range_t`
- Add support for nested containers
- Remove syntax without parenthesis

Revision 2

- Remove the implicit `const` removal when converting an associative container to a container of pairs
- Use CTAD to determine the value type of the returned container
- Attempt at wording

Revision 1

- Split out the proposed constructors for string view and span into separate papers ([P1391] and [?]) respectively)
- Use a function based approach rather than adding a constructor to standard containers, as it proved unworkable.

Overview

We propose 2 facilities to make it easier to construct container and from ranges:

- `ranges::to` a function that can materialize any range as a container, including non-standard containers, and recursive containers
- tagged constructors, `insert` and `assign` methods for standard containers and string types.

We propose that all the following syntaxes be valid constructs. The examples are meant to illustrate the interface's capabilities - the primary use case for it is to materialize views, even if copying from one container type to another is possible.

```
auto l = std::views::iota(1, 10);

// create a vector with the elements of l
auto vec = ranges::to<std::vector<int>>(l); // or vector{std::from_range, l};

//Specify an allocator
auto b = ranges::to<std::vector<int, Alloc>>(l, alloc); // or vector{std::from_range, l, alloc};

//deducing value_type
auto c = ranges::to<std::vector>(l);

// explicit conversion int -> long
auto d = ranges::to<std::vector<long>>(l);

//Supports converting associative container to sequence containers
auto f = ranges::to<vector>(m);

//Supports converting sequence containers to associative ones
auto g = ranges::to<map>(f);

//Pipe syntaxe
auto g = l | ranges::view::take(42) | ranges::to<std::vector>();

//Pipe syntax with allocator
auto h = l | ranges::view::take(42) | ranges::to<std::vector>(alloc);

//The pipe syntax also support specifying the type and conversions
auto i = l | ranges::view::take(42) | ranges::to<std::vector<long>>();
```

```
// Nested ranges
std::list<std::forward_list<int>> lst = {{0, 1, 2, 3}, {4, 5, 6, 7}};
auto vec1 = ranges::to<std::vector<std::vector<int>>>(lst);
auto vec2 = ranges::to<std::vector<std::deque<double>>>(lst);
```

Tony tables

Before	After
<pre>std::list<int> lst = /*...*/; std::vector<int> vec {std::begin(lst), std::end(lst)};</pre>	<pre>std::vector<int> vec = lst ranges::to<std::vector>();</pre>
<pre>auto view = ranges::iota(42); vector < iter_value_t< iterator_t<decltype(view)> > > vec; if constexpr(SizedRanged<decltype(view)>) { vec.reserve(ranges::size(view)); } ranges::copy(view, std::back_inserter(vec));</pre>	<pre>auto vec = ranges::iota(0, 42) ranges::to<std::vector>();</pre>
<pre>std::map<int, widget> map = get_widgets_map(); std::vector< typename decltype(map)::value_type > vec; vec.reserve(map.size()); ranges::move(map, std::back_inserter(vec));</pre>	<pre>auto vec = get_widgets_map() ranges::to<vector>();</pre>

Design Notes

from_range is declared as an instance of a tag_type from_range_t in std.

ranges::to

ranges::to<C>(r, args) returns an instance c of C constructed by the first valid method among the following:

- Construct `c` from `r`
- Construct `c` from `r` using the tagged ranged constructor (`from_range_t`)
- Construct `c` from the pair of iterators `ranges::begin(r)`, `ranges::end(r)`
- Construct `c`, then insert each element of `r` at the end of `c`.
- If `C` is a range whose value type is itself a range (and is not a view), and `r`'s value type is also a range, the application of `to<range_value_t<C>>` for each element of `r` is inserted at the end of `c`.

Any additional arguments passed to `to` are forwarded as the last parameters of the constructor, in order to support allocators. This also allow passing comparators, hasher, etc to containers when applicable.

Containers range constructors and methods

For any constructor or methods taking a pair of InputIterators in containers (with the exception of `regex` and `filesystem::path`), a similar method, suffixed with `_range` is added taking a range instead.

In addition, by SG9 request. `push_front_range` and `push_back_range` are added when an equivalent `push_front` or `push_back` exists.

prepend_range and append_range both preserve the order of elements. Notably, `prepend_range` does not reverse the order of the elements in the inserted range. Both because we think this would be surprising for most people and because it would require `bidirectional_range`, which would make it less useful. It is possible to use `reverse_view` to reverse the elements of the inserted range, which seems like the natural way to express that intent.

All added constructors are tagged with `from_range_t`. Methods that may be ambiguous are suffixed with `_range`.

The container's value type must be explicitly constructible from the reference type of the `input_range` Range.

The following methods and constructors are added to all sequence containers (`vector`, `deque`, `list`, `forward_list`):

- `Container(from_range_t, Range&&, const Allocator& = {});`
- `iterator insert_range(const_iterator position, Range&&);` (`insert_after_range` in `forward_list`)

`vector`, `deque`, `list`, `vector` gain:

- `void append_range(Range&&);`

`deque`, `forward_list`, `list` gain:

- `void prepend_range(Range&&);`

`basic_string` gains:

- `basic_string(from_range_t, Range&&, const Allocator& = {});`
- `iterator insert_range(const_iterator position, Range&&);`
- `basic_string& assign_range(Range&&);`
- `basic_string& replace_range(const_iterator, const_iterator, Range&&);`
- `basic_string& append_range(Range&&);`

The following methods and constructors are added to associative containers (`set`, `multiset`, `map`, `multimap`)

- `Container(from_range_t, Range&&, const Compare& = {}, const Allocator& = {});`
- `void insert_range(Range&&);`

The following methods and constructors are added to unordered containers (`unordered_set`, `unordered_multiset`, `unordered_map`, `unordered_multimap`):

- `Container(from_range_t, Range&&, size_t n = /**/, const hasher& = {}, const key_equal& = {}, const Allocator& = {});`
- `Container(from_range_t, Range&&, size_t n, const Allocator&);`
- `Container(from_range_t, Range&&, size_t n, const hasher&, const Allocator&);`
- `void insert_range(Range&&);`

`priority_queue` gains:

- `priority_queue(from_range_t, Range&&, const Compare& = {});`
- `priority_queue(from_range_t, Range&&, const Compare, const Alloc&);`
- `priority_queue(from_range_t, Range&&, const Alloc&);`
- `void push_range(Range&&);`

`stack` and `queue` gain:

- `Container(from_range_t, Range&&);`
- `Container(from_range_t, Range&&, const Alloc&);`
- `void push_range(Range&&);`

Note that for `stack`, `queue`, `priority_queue`, `push_range` do not requires an equivalent `append_range` method to exists.

For every constructor, a deduction guide is added.

Considerations

Why do we need this?

Containers do not have containers constructors, so

```
vector v = views::iota(0, 10);
```

is currently not valid syntax.

They do have a constructors taking a pair of iterators. So, it would theoretically be possible to write:

```
auto view = views::iota(0, 10);  
vector<int> v(ranges::begin(view), ranges::end(view));
```

Which is more cumbersome. But that isn't enough! Containers expect the same types for both iterators - they do not support sentinels. A solution would be to write:

```
auto view = views::iota(0, 10) | views::common;  
vector<int> v(ranges::begin(view), ranges::end(view));
```

And that still does not always work. 'Cpp17Iterators' required by containers can have slightly different semantics. Namely, 'input_iterator' may not be copyable. So the following does not work:

```
std::generator<std::pair<int, string>> f() {  
    co_yield {0, "Hello"};  
    co_yield {1, "World"};  
}  
auto view = f(); // attempts to use views::common here would be ill-formed  
map<int, std::string> v(ranges::begin(view), ranges::end(view));
```

Instead, one has to insert each element manually.

This is sufficiently complex and error-prone approach that multiple blog posts and stackoverflow questions address it:

- [How to make a container from a C++20 range](#)
- [Will we be able to construct containers with views in C++20?](#)
- [Range concept and containers constructors](#)
- [Initializing std::vector with ranges library](#)

Why do we need a tag / different method names ?

Ambiguities can arise in 2 cases. Using different methods resolves these ambiguities.

CTAD

Consider the following code:

```
std::list<int> l;  
std::vector v{1};
```

Should `v` be `std::vector<int>` or `std::vector<std::list<int>>`? It is currently equivalent to the latter! Adding a tag solves this issue(although one needs to remember using a tag!).

Ambiguous conversions

There are other issues with `vector<any>`.

```
std::list<any> l;  
std::vector<any> v;  
v.insert(1);
```

Does that insert a range of any or a single any? Using a different name resolves this ambiguity. `assign` always takes a range or count + value, so it does not suffer this ambiguity.

Do we need both approaches?

Both approaches are complementary. `ranges::to` works with non-standard containers and supports constructing containers of containers. Tagged constructors offer an opportunity for containers to provide a more efficient implementation. For example, many containers do not have a `reserve` method. `ranges::to` uses the tagged constructor when available. As such, tagged constructors offer a customization mechanism to opt-in into `ranges::to` and could be extended to things that are not containers.

`ranges::to` does not replace the proposed `insert_range` and `assign_range` methods.

While we recommend pursuing both approaches, it is essential to make sure `ranges::to` is part of C++23 as it is a critical missing piece of ranges. The two approaches can easily be split off if necessary.

Performance Consideration

In many cases, a container can implement range construction more efficiently than ranges can, notably, because it has access to the initialized memory.

We can demonstrate that by comparing 3 scenarios:

Iterator construction

```
auto c = range | views::common;  
std::vector<T> v(c.begin(), c.end());
```

reserve + copy (similar to `ranges::to` in the absence of tags)


```
std::vector<T*> v;
v.reserve(std::ranges::distance(range));
std::ranges::copy(range, std::back_inserter(v));
```

using from_range

```
std::vector vector(std::ranges::from_range, range);
```

Barry Revzin tested different scenarios and reported the following numbers:

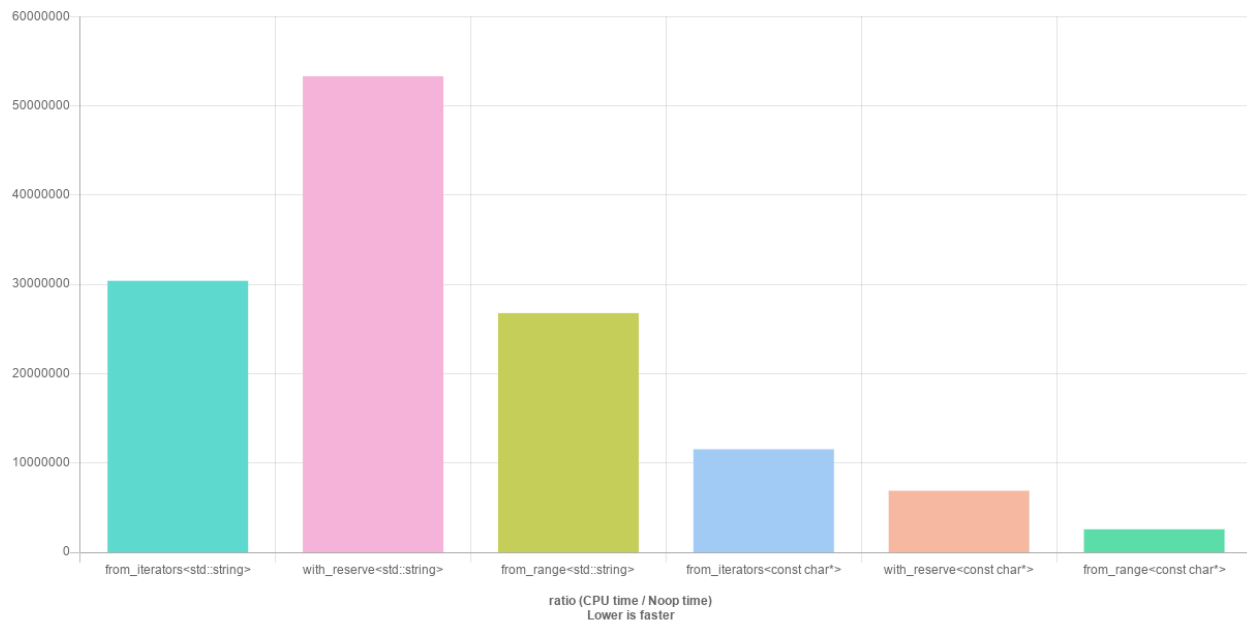
A A random access, sized, common range of prvalue char const* (so, an input range in C++17) constructing a vector<string>.

B A random access, non-sized, non-common range of lvalue string (so, a forward range in C++17) constructing a vector<string>

C A random access, sized, common range of prvalue char const* constructing a vector<char const*> (so, the individual element construction is very cheap).

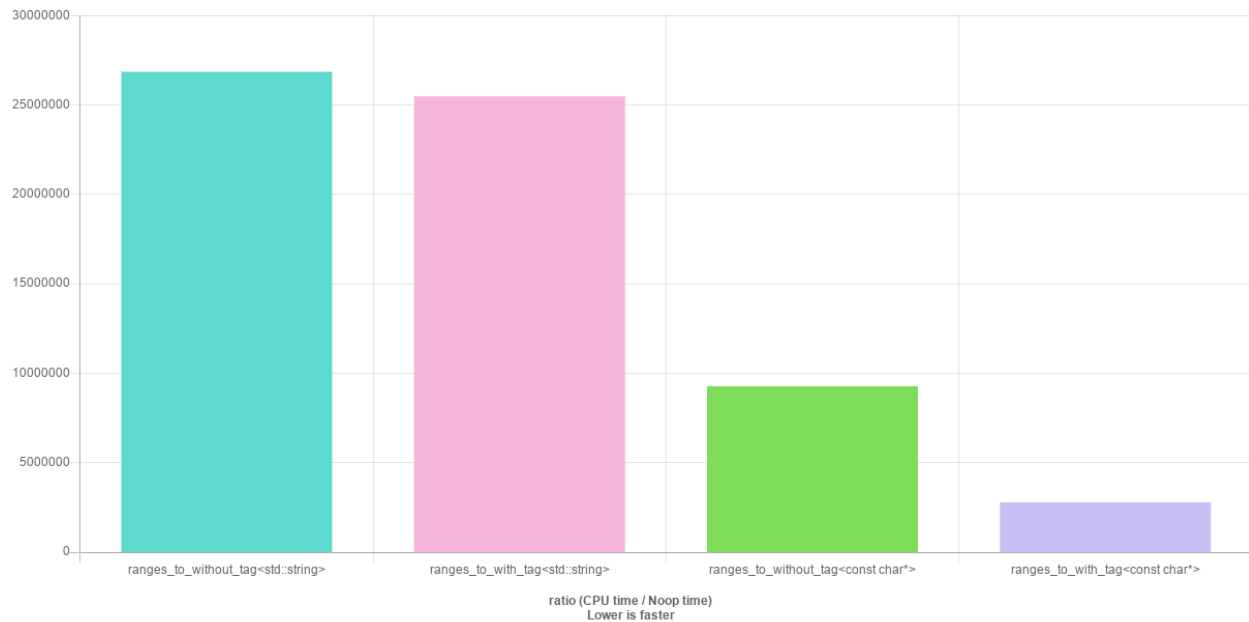
	iterators constructor	reserve + copy	from_range_t constructor
A	90.9µs	57.0µs	52.3µs
B	79.4µs	69.7µs	67.9µs
C	3.7µs	4.8µs	1.8µs

I've reproduced scenarios A&C on [QuickBench](#)



In both scenarios, the from_range constructor performs significantly better than both the iterator pair constructor and the reserve + copy method employed by ranges::to

Because ranges::to will call a tag constructor when there is one, it shows the same difference.



QuickBench

Do we need iterator/sentinel pair constructors as well?

The new standard library algorithms in the `'std::ranges'` namespace all provide multiple overloads taking both a range and an iterator/sentinel pair. This raises the question: should our new constructors provide a "range" overload (only), an iterator/sentinel pair overload (only), or both?

The second case (an iterator/sentinel pair only) is easily dismissed. There exist ranges which model `sized_range`, but whose iterators/sentinels do not model `sized_sentinel_for` - the canonical example being `std::list`. With such types, a constructor call like

```
std::list<int> list = get_list();
std::vector<int> vec(std::from_range, list); // copy into vector
```

would know the size of the list and would therefore be able to allocate the required vector size upfront. On the other hand, a hypothetical call to

```
std::list<int> list = get_list();
std::vector<int> vec(std::from_range, list.begin(), list.end());
```

cannot know the size of the input range upfront, and so would need two passes over the data.

We are thus left to decide whether to provide an iterator/sentinel constructor in addition to a "range" constructor. In the authors' opinion, this would be redundant. In cases where we do have separate iterator and sentinel objects that we wish to pass to a container constructor, we can do so using 'subrange', as in

```
std::vector vec(std::from_range, subrange(my_iter, my_sentinel));
```

Furthermore, it is intended that users should be able to opt-in to `ranges::to` support for their own container classes by providing constructors which take `from_range_t` as their first argument. Adding iterator/sentinel constructors in addition to "range" constructors means more work for users to adhere to the protocol, and no doubt risks confusion about whether one, either, or both are required.

***reservable-container* concept**

The exposition only *reservable-container* concepts checks for

- `reserve`
- `max_size`
- `capacity`

which is consistent with `ranges-v3`. This serves multiple purposes:

- Avoid calling `reserve` on unordered containers and others entities that have a `reserve` method with different semantics
- Allow implementations to check that we are not trying to reserve more than `max_size`

Implementation Experience

Implementations of `ranges::to` are available in [[RangeV3](#)], [[cmcstl2](#)] and on Github [[rangesnext](#)]. The tagged ranges constructors, insert methods and other range-taking container members fuctions have **not** been implemented.

Related Paper and future work

Future work is needed to allow constructing `std::array` from *tiny-ranges*. This paper also do not proposes modifications of `path`(the author is not familiar with it enough) and `regex` (whose fate is in the balance).

Previous polls

SG9 POLL: All range-based member functions should end with "`_range`" (e.g. `assign_range`, `insert_range`, `append_range`) except for the constructors which take tags.

F	N	A
7	1	0

SG9 POLL: We would like to add `push_back_range`, `push_front_range` to the relevant containers in the paper

F	N	A
6	3	0

SG9 POLL: We approve the direction and design of D1206r4 with the recommended changes, and recommend forwarding it to LEWG

F	N	A
9	0	0

Proposed wording

Wording is relative to [\[N4885\]](#).

❖ **Containers** [containers]

❖ **Container requirements** [container.requirements]

❖ **General container requirements** [container.requirements.general]

- *T* is *Cpp17EmplaceConstructible into X from args*, for zero or more arguments *args*, means that the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, args)
```

- *T* is *Cpp17Erasable from X* means that the following expression is well-formed:

```
allocator_traits<A>::destroy(m, p)
```

[*Note*: A container calls `allocator_traits<A>::construct(m, p, args)` to construct an element at *p* using *args*, with `m == get_allocator()`. The default construct in allocator will call `::new((void*)p) T(args)`, but specialized allocators can choose a different definition. — *end note*]

The following exposition-only concept is used in the definition of containers:

```
template<class R, class T>
concept container-compatible-range = // exposition only
    ranges::input_range<R> &&
    convertible_to<ranges::range_reference_t<R>, T>;
```

❖ **Sequence containers** [sequence.reqmts]

In Tables [\[tab:container.seq.req\]](#) and [\[tab:container.seq.opt\]](#),

- *X* denotes a sequence container class,
- *a* denotes a value of type *X* containing elements of type *T*,

- u denotes the name of a variable being declared,
- A denotes $X::\text{allocator_type}$ if the *qualified-id* $X::\text{allocator_type}$ is valid and denotes a type and $\text{allocator}<T>$ if it doesn't,
- i and j denote iterators that meet the *Cpp17InputIterator* requirements and refer to elements implicitly convertible to value_type ,
- $[i, j)$ denotes a valid range,
- rg denotes a value of a type R that models *container-compatible-range* $<T>$,
- il designates an object of type $\text{initializer_list}<\text{value_type}>$,
- n denotes a value of type $X::\text{size_type}$,
- p denotes a valid constant iterator to a ,
- q denotes a valid dereferenceable constant iterator to a ,
- $[q1, q2)$ denotes a valid range of constant iterators in a ,
- t denotes an lvalue or a const rvalue of $X::\text{value_type}$, and
- rv denotes a non-const rvalue of $X::\text{value_type}$.
- Args denotes a template parameter pack;
- args denotes a function parameter pack with the pattern $\text{Args}\&\&$.

The complexities of the expressions are sequence dependent.

Table 1: Sequence container requirements (in addition to container)

Expression	Return type	Assertion/note pre-/post-condition
$X(n, t)$ $X u(n, t);$		<i>Preconditions:</i> T is <i>Cpp17CopyInsertable</i> into X . <i>Postconditions:</i> $\text{distance}(\text{begin}(), \text{end}()) == n$ <i>Effects:</i> Constructs a sequence container with n copies of t

Table 1: Sequence container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition
<code>X(i, j)</code> <code>X u(i, j);</code>		<i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into x from *i. For vector, if the iterator does not meet the <i>Cpp17ForwardIterator</i> requirements, T is also <i>Cpp17MoveInsertable</i> into x. <i>Postconditions:</i> <code>distance(begin(), end()) == distance(i, j)</code> <i>Effects:</i> Constructs a sequence container equal to the range [i, j). Each iterator in the range [i, j) is dereferenced exactly once.
<code>X(from_range, rg)</code>		<i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into x from <code>*ranges::begin(rg)</code> . For vector, if R models neither <i>sized_range</i> nor <i>forward_range</i> , T is also <i>Cpp17MoveInsertable</i> into x. <i>Effects:</i> Constructs a sequence container equal to the range rg. Each iterator in the range rg is dereferenced exactly once. <i>Postconditions:</i> <code>distance(begin(), end()) == ranges::distance(rg)</code>
<code>X(il)</code>		Equivalent to <code>X(il.begin(), il.end())</code>
<code>a = il</code>	<code>X&</code>	<i>Preconditions:</i> T is <i>Cpp17CopyInsertable</i> into x and <i>Cpp17CopyAssignable</i> . <i>Effects:</i> Assigns the range [il.begin(), il.end()) into a. All existing elements of a are either assigned to or destroyed. <i>Returns:</i> *this.
<code>a.emplace(p, args)</code>	iterator	<i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into x from args. For vector and deque, T is also <i>Cpp17MoveInsertable</i> into x and <i>Cpp17MoveAssignable</i> . <i>Effects:</i> Inserts an object of type T constructed with <code>std::forward<Args>(args)...</code> before p.

Table 1: Sequence container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition
<code>a.insert(p, t)</code>	iterator	<i>Preconditions:</i> T is <i>Cpp17CopyInsertable</i> into X. For vector and deque, T is also <i>Cpp17CopyAssignable</i> . <i>Effects:</i> Inserts a copy of t before p.
<code>a.insert(p, rv)</code>	iterator	<i>Preconditions:</i> T is <i>Cpp17MoveInsertable</i> into X. For vector and deque, T is also <i>Cpp17MoveAssignable</i> . <i>Effects:</i> Inserts a copy of rv before p.
<code>a.insert(p, n, t)</code>	iterator	<i>Preconditions:</i> T is <i>Cpp17CopyInsertable</i> into X and <i>Cpp17CopyAssignable</i> . <i>Effects:</i> Inserts n copies of t before p.
<code>a.insert(p, i, j)</code>	iterator	<i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into X from *i. For vector and deque, T is also <i>Cpp17MoveInsertable</i> into X, <i>Cpp17MoveConstructible</i> , <i>Cpp17MoveAssignable</i> , and swappable. Neither i nor j are iterators into a. <i>Effects:</i> Inserts copies of elements in [i, j) before p. Each iterator in the range [i, j) shall be dereferenced exactly once.
<code>a.insert_range(p, rg)</code>	iterator	<i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into X from * ranges::begin(rg) . For vector and deque, T is also <i>Cpp17MoveInsertable</i> into X, <i>Cpp17MoveConstructible</i> , <i>Cpp17MoveAssignable</i> , and swappable. rg and a do not overlap. <i>Effects:</i> Inserts copies of elements in rg before p. Each iterator in the range rg is dereferenced exactly once.
<code>X(il)</code>		Equivalent to <code>X(il.begin(), il.end())</code>
<code>a.insert(p, il)</code>	iterator	<code>a.insert(p, il.begin(), il.end())</code> .
<code>a.erase(q)</code>	iterator	<i>Preconditions:</i> For vector and deque, T is <i>Cpp17MoveAssignable</i> . <i>Effects:</i> Erases the element pointed to by q.

Table 1: Sequence container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition
<code>a.erase(q1,q2)</code>	iterator	<i>Preconditions:</i> For vector and deque, T is <i>Cpp17MoveAssignable</i> . <i>Effects:</i> Erases the elements in the range [q1, q2).
<code>a.clear()</code>	void	<i>Effects:</i> Destroys all elements in a. Invalidates all references, pointers, and iterators referring to the elements of a and may invalidate the past-the-end iterator. <i>Postconditions:</i> <code>a.empty()</code> is true. <i>Complexity:</i> Linear.
<code>a.assign(i,j)</code>	void	<i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into x from *i and assignable from *i. For vector, if the iterator does not meet the forward iterator requirements, T is also <i>Cpp17MoveInsertable</i> into x. Neither i nor j are iterators into a. <i>Effects:</i> Replaces elements in a with a copy of [i, j). Invalidates all references, pointers and iterators referring to the elements of a. For vector and deque, also invalidates the past-the-end iterator. Each iterator in the range [i, j) shall be dereferenced exactly once.

Table 1: Sequence container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition
<code>a.assign_range(rg)</code>	<code>void</code>	<i>Mandates:</i> <code>assignable_from<T&, ranges::range_reference_t<R>></code> is modeled. <i>Preconditions:</i> <code>T</code> is <i>Cpp17EmplaceConstructible</i> into <code>x</code> from <code>*ranges::begin(rg)</code>. For <code>vector</code>, if <code>R</code> models neither <code>sized_range</code> nor <code>forward_range</code>, <code>T</code> is also <i>Cpp17MoveInsertable</i> into <code>x</code>. <code>rg</code> and <code>a</code> do not overlap. <i>Effects:</i> Replaces elements in <code>a</code> with a copy of each element in <code>rg</code>. Invalidates all references, pointers and iterators referring to the elements of <code>a</code>. For <code>vector</code> and <code>deque</code>, also invalidates the past-the-end iterator. Each iterator in the range <code>rg</code> is dereferenced exactly once.
<code>a.assign(il)</code>	<code>void</code>	<code>a.assign(il.begin(), il.end())</code> .
<code>a.assign(n,t)</code>	<code>void</code>	<i>Preconditions:</i> <code>T</code> is <i>Cpp17CopyInsertable</i> into <code>x</code> and <i>Cpp17CopyAssignable</i>. <code>t</code> is not a reference into <code>a</code>. <i>Effects:</i> Replaces elements in <code>a</code> with <code>n</code> copies of <code>t</code>. Invalidates all references, pointers and iterators referring to the elements of <code>a</code>. For <code>vector</code> and <code>deque</code>, also invalidates the past-the-end iterator.

The iterator returned from `a.insert(p, t)` points to the copy of `t` inserted into `a`.

The iterator returned from `a.insert(p, rv)` points to the copy of `rv` inserted into `a`.

The iterator returned from `a.insert(p, n, t)` points to the copy of the first element inserted into `a`, or `p` if `n == 0`.

The iterator returned from `a.insert(p, i, j)` points to the copy of the first element inserted into `a`, or `p` if `i == j`.

The iterator returned from `a.insert_range(p, rg)` points to the copy of the first element inserted into `a`, or `p` if `rg` is empty.

The iterator returned from `a.insert(p, i1)` points to the copy of the first element inserted into `a`, or `p` if `i1` is empty.

The iterator returned from `a.emplace(p, args)` points to the new element constructed from `args` into `a`.

The iterator returned from `a.erase(q)` points to the element immediately following `q` prior to the element being erased. If no such element exists, `a.end()` is returned.

The iterator returned by `a.erase(q1, q2)` points to the element pointed to by `q2` prior to any elements being erased. If no such element exists, `a.end()` is returned.

For every sequence container defined in this Clause and in ??:

- If the constructor

```
template<class InputIterator>
X(InputIterator first, InputIterator last,
  const allocator_type& alloc = allocator_type());
```

is called with a type `InputIterator` that does not qualify as an input iterator, then the constructor shall not participate in overload resolution.

- If the member functions of the forms:

```
template<class InputIterator>
return-type F(const_iterator p,
InputIterator first, InputIterator last);          // such as insert

template<class InputIterator>
return-type F(InputIterator first, InputIterator last);          // such as append, assign

template<class InputIterator>
return-type F(const_iterator i1, const_iterator i2,
InputIterator first, InputIterator last);          // such as replace
```

are called with a type `InputIterator` that does not qualify as an input iterator, then these functions shall not participate in overload resolution.

- A deduction guide for a sequence container shall not participate in overload resolution if it has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter, or if it has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.

lists operations that are provided for some types of sequence containers but not others. An implementation shall provide these operations for all container types shown in the “container” column, and shall implement them so as to take amortized constant time.

Table 2: Optional sequence container operations

Expression	Return type	Operational semantics	Container
a.front()	reference; const_reference for constant a	*a.begin()	basic_ string, array, deque, forward_ list, list, vector
a.back()	reference; const_reference for constant a	{ auto tmp = a.end(); --tmp; return *tmp; }	basic_ string, array, deque, list, vector
a.emplace_ front(args)	reference	<i>Effects:</i> Prepends an object of type T constructed with std::forward<Args>(args).... <i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into X from args. <i>Returns:</i> a.front().	deque, forward_ list, list
a.emplace_ back(args)	reference	<i>Effects:</i> Appends an object of type T constructed with std::forward<Args>(args).... <i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into X from args. For vector, T is also <i>Cpp17MoveInsertable</i> into x. <i>Returns:</i> a.back().	deque, list, vector
a.push_ front(t)	void	<i>Effects:</i> Prepends a copy of t. <i>Preconditions:</i> T is <i>Cpp17CopyInsertable</i> into x.	deque, forward_ list, list
a.push_ front(rv)	void	<i>Effects:</i> Prepends a copy of rv. <i>Preconditions:</i> T is <i>Cpp17MoveInsertable</i> into x.	deque, forward_ list, list

Table 2: Optional sequence container operations (continued)

Expression	Return type	Operational semantics	Container
<code>a.prepend_range(rg)</code>	<u>void</u>	<i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into X from <i>*ranges::begin(rg)</i> . <i>Effects:</i> Inserts copies of elements in rg before <i>begin()</i> . Each iterator in the range rg is dereferenced exactly once. [<i>Note:</i> The order of elements in rg is not reversed. — <i>end note</i>]	<u>deque</u> , <u>forward_list</u> , <u>list</u>
<code>a.push_back(t)</code>	void	<i>Effects:</i> Appends a copy of t. <i>Preconditions:</i> T is <i>Cpp17CopyInsertable</i> into X.	<code>basic_string</code> , <code>deque</code> , <code>list</code> , <code>vector</code>
<code>a.push_back(rv)</code>	void	<i>Effects:</i> Appends a copy of rv. <i>Preconditions:</i> T is <i>Cpp17MoveInsertable</i> into X.	<code>basic_string</code> , <code>deque</code> , <code>list</code> , <code>vector</code>
<code>a.append_range(rg)</code>	<u>void</u>	<i>Preconditions:</i> T is <i>Cpp17EmplaceConstructible</i> into X from <i>*ranges::begin(rg)</i> . For <code>vector</code> , T is also <i>Cpp17MoveInsertable</i> into X. <i>Effects:</i> Inserts copies of elements in rg before <i>end()</i> . Each iterator in the range rg is dereferenced exactly once.	<u>deque</u> , <u>list</u> , <u>vector</u>
<code>a.pop_front()</code>	void	<i>Effects:</i> Destroys the first element. <i>Preconditions:</i> <code>a.empty()</code> is false.	<code>deque</code> , <code>forward_list</code> , <code>list</code>
<code>a.pop_back()</code>	void	<i>Effects:</i> Destroys the last element. <i>Preconditions:</i> <code>a.empty()</code> is false.	<code>basic_string</code> , <code>deque</code> , <code>list</code> , <code>vector</code>

Table 2: Optional sequence container operations (continued)

Expression	Return type	Operational semantics	Container
<code>a[n]</code>	reference; const_reference for constant a	<code>*(a.begin() + n)</code>	basic_ string, array, deque, vector
<code>a.at(n)</code>	reference; const_reference for constant a	<code>*(a.begin() + n)</code>	basic_ string, array, deque, vector

The member function `at()` provides bounds-checked access to container elements. `at()` throws `out_of_range` if `n >= a.size()`.

◆ Associative containers [associative.reqmts]

◆ General [associative.reqmts.general]

Associative containers provide fast retrieval of data based on keys. The library provides four basic kinds of associative containers: `set`, `multiset`, `map` and `multimap`.

Each associative container is parameterized on `Key` and an ordering relation `Compare` that induces a strict weak ordering on elements of `Key`. In addition, `map` and `multimap` associate an arbitrary *mapped type* `T` with the `Key`. The object of type `Compare` is called the *comparison object* of a container.

The phrase “equivalence of keys” means the equivalence relation imposed by the comparison object. That is, two keys `k1` and `k2` are considered to be equivalent if for the comparison object `comp`, `comp(k1, k2) == false && comp(k2, k1) == false`. [Note: This is not necessarily the same as the result of `k1 == k2`. —end note] For any two keys `k1` and `k2` in the same container, calling `comp(k1, k2)` shall always return the same value.

An associative container supports *unique keys* if it may contain at most one element for each key. Otherwise, it supports *equivalent keys*. The `set` and `map` classes support unique keys; the `multiset` and `multimap` classes support equivalent keys. For `multiset` and `multimap`, `insert`, `emplace`, and `erase` preserve the relative ordering of equivalent elements.

For `set` and `multiset` the value type is the same as the key type. For `map` and `multimap` it is equal to `pair<const Key, T>`.

iterator of an associative container is of the bidirectional iterator category. For associative containers where the value type is the same as the key type, both `iterator` and `const_iterator` are constant iterators. It is unspecified whether or not `iterator` and `const_iterator` are the same type. [Note: `iterator` and `const_iterator` have identical semantics in this case, and `iterator` is convertible to `const_iterator`. Users can avoid violating the one-definition rule by always using `const_iterator` in their function parameter lists. —end note]

The associative containers meet all the requirements of Allocator-aware containers, except that for map and multimap, the requirements placed on value_type in apply instead to key_type and mapped_type. [Note: For example, in some cases key_type and mapped_type are required to be Cpp17CopyAssignable even though the associated value_type, pair<const key_type, mapped_type>, is not Cpp17CopyAssignable. — end note]

In ,

- X denotes an associative container class,
- a denotes a value of type X,
- a2 denotes a value of a type with nodes compatible with type X (),
- b denotes a possibly const value of type X,
- u denotes the name of a variable being declared,
- a_uniq denotes a value of type X when X supports unique keys,
- a_eq denotes a value of type X when X supports multiple keys,
- a_tran denotes a possibly const value of type X when the *qualified-id* X::key_compare::is_transparent is valid and denotes a type,
- i and j meet the Cpp17InputIterator requirements and refer to elements implicitly convertible to value_type,
- [i, j) denotes a valid range,
- rg denotes a value of a type R that models container-compatible-range<value_type>,
- p denotes a valid constant iterator to a,
- q denotes a valid dereferenceable constant iterator to a,
- r denotes a valid dereferenceable iterator to a,
- [q1, q2) denotes a valid range of constant iterators in a,
- il designates an object of type initializer_list<value_type>,
- t denotes a value of type X::value_type,
- k denotes a value of type X::key_type, and
- c denotes a possibly const value of type X::key_compare;
- k1 is a value such that a is partitioned with respect to c(r, k1), with r the key value of e and e in a;
- ku is a value such that a is partitioned with respect to !c(ku, r);
- ke is a value such that a is partitioned with respect to c(r, ke) and !c(ke, r), with c(r, ke) implying !c(ke, r).
- A denotes the storage allocator used by X, if any, or allocator<X::value_type> otherwise,

- `m` denotes an allocator of a type convertible to `A`, and `nh` denotes a non-const rvalue of type `X::node_type`.

Table 3: Associative container requirements (in addition to container)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>X(i,j,c)</code> <code>X u(i,j,c);</code>		<i>Preconditions:</i> <code>value_type</code> is <code>Cpp17EmplaceConstructible</code> into <code>X</code> from <code>*i</code> . <i>Effects:</i> Constructs an empty container and inserts elements from the range <code>[i, j)</code> into it; uses <code>c</code> as a comparison object.	$N \log N$ in general, where N has the value <code>distance(i, j)</code> ; linear if <code>[i, j)</code> is sorted with <code>value_comp()</code>
<code>X(i,j)</code> <code>X u(i,j);</code>		<i>Preconditions:</i> <code>key_compare</code> meets the <code>Cpp17DefaultConstructible</code> requirements. <code>value_type</code> is <code>Cpp17EmplaceConstructible</code> into <code>X</code> from <code>*i</code> . <i>Effects:</i> Same as above, but uses <code>Compare()</code> as a comparison object.	same as above
<code>X(from_range, rg, c)</code>		<i>Preconditions:</i> <code>value_type</code> is <code>Cpp17EmplaceConstructible</code> into <code>X</code> from <code>*ranges::begin(rg)</code> . <i>Effects:</i> Constructs an empty container and inserts each element from <code>rg</code> into it. Uses <code>c</code> as the comparison object.	$N \log N$ in general, where N has the value <code>ranges::distance(rg)</code> ; linear if <code>rg</code> is sorted with <code>value_comp()</code>

Table 3: Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>X(from_range, rg)</code>		<i>Preconditions:</i> <code>key_compare</code> meets the <i>Cpp17DefaultConstructible</i> requirements. <code>value_type</code> is <i>Cpp17EmplaceConstructible</i> into <code>X</code> from <code>*ranges::begin(rg)</code> . <i>Effects:</i> Constructs an empty container and inserts each element from <code>rg</code> into it. Uses <code>Compare()</code> as the comparison object.	same as above
<code>X(il)</code>		same as <code>X(il.begin(), il.end())</code>	same as <code>X(il.begin(), il.end())</code>
<code>X(il, c)</code>		same as <code>X(il.begin(), il.end(), c)</code>	same as <code>X(il.begin(), il.end(), c)</code>
<code>a = il</code>	<code>X&</code>	<i>Preconditions:</i> <code>value_type</code> is <i>Cpp17CopyInsertable</i> into <code>X</code> and <i>Cpp17CopyAssignable</i> . <i>Effects:</i> Assigns the range <code>[il.begin(), il.end())</code> into <code>a</code> . All existing elements of <code>a</code> are either assigned to or destroyed.	$N \log N$ in general, where N has the value <code>il.size() + a.size()</code> ; linear if <code>[il.begin(), il.end())</code> is sorted with <code>value_comp()</code>
<code>a.insert(i, j)</code>	<code>void</code>	<i>Preconditions:</i> <code>value_type</code> is <i>Cpp17EmplaceConstructible</i> into <code>X</code> from <code>*i</code> . Neither <code>i</code> nor <code>j</code> are iterators into <code>a</code> . <i>Effects:</i> Inserts each element from the range <code>[i, j)</code> if and only if there is no element with key equivalent to the key of that element in containers with unique keys; always inserts that element in containers with equivalent keys.	$N \log(a.size() + N)$, where N has the value <code>distance(i, j)</code>

Table 3: Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
a.insert_range(rg) void		<i>Preconditions:</i> <code>value_type</code> is <code>Cpp17EmplaceConstructible</code> into <code>x</code> from <code>*ranges::begin(rg).rg</code> and <code>a</code> do not overlap. <i>Effects:</i> Inserts each element from <code>rg</code> if and only if there is no element with key equivalent to the key of that element in containers with unique keys; always inserts that element in containers with equivalent keys.	$N \log(a.size() + N)$, where N has the value <code>ranges::distance(rg)</code>

The `insert`, [insert_range](#), and `emplace` members shall not affect the validity of iterators and references to the container, and the `erase` members shall invalidate only iterators and references to the erased elements.

The `extract` members invalidate only iterators to the removed element; pointers and references to the removed element remain valid. However, accessing the element through such pointers and references while the element is owned by a `node_type` is undefined behavior. References and pointers to an element obtained while it is owned by a `node_type` are invalidated if the element is successfully inserted.

The fundamental property of iterators of associative containers is that they iterate through the containers in the non-descending order of keys where non-descending is defined by the comparison that was used to construct them. For any two dereferenceable iterators `i` and `j` such that distance from `i` to `j` is positive, the following condition holds:

```
value_comp(*j, *i) == false
```

For associative containers with unique keys the stronger condition holds:

```
value_comp(*i, *j) != false
```

When an associative container is constructed by passing a comparison object the container shall not store a pointer or reference to the passed object, even if that object is passed by reference. When an associative container is copied, through either a copy constructor or an assignment operator, the target container shall then use the comparison object from the container being copied, as if that comparison object had been passed to the target container in its constructor.

The member function templates `find`, `count`, `contains`, `lower_bound`, `upper_bound`, and `equal_range` shall not participate in overload resolution unless the *qualified-id* `Compare::is_transparent` is valid and denotes a type.

A deduction guide for an associative container shall not participate in overload resolution if any of the following are true:

- It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
- It has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.
- It has a `Compare` template parameter and a type that qualifies as an allocator is deduced for that parameter.

◆ General

[unord.req.general]

// ...

In ,

- `X` denotes an unordered associative container class,
- `a` denotes a value of type `X`,
- `a2` denotes a value of a type with nodes compatible with type `X()`,
- `b` denotes a possibly const value of type `X`,
- `a_uniq` denotes a value of type `X` when `X` supports unique keys,
- `a_eq` denotes a value of type `X` when `X` supports equivalent keys,
- `a_tran` denotes a possibly const value of type `X` when the *qualified-id* `X::key_equal::is_transparent` and `X::hasher::is_transparent` are both valid and denote types,
- `i` and `j` denote input iterators that refer to `value_type`,
- `[i, j)` denotes a valid range,
- `rg` denotes a value of a type `R` that models *container-compatible-range*`<value_type>`,
- `p` and `q2` denote valid constant iterators to `a`,
- `q` and `q1` denote valid dereferenceable constant iterators to `a`,
- `r` denotes a valid dereferenceable iterator to `a`,
- `[q1, q2)` denotes a valid range in `a`,
- `il` denotes a value of type `initializer_list<value_type>`,
- `t` denotes a value of type `X::value_type`,
- `k` denotes a value of type `key_type`,

- hf denotes a possibly const value of type hasher,
 - eq denotes a possibly const value of type key_equal,
 - ke is a value such that
 - eq(r1, ke) == eq(ke, r1)
 - hf(r1) == hf(ke) if eq(r1, ke) is true, and
 - (eq(r1, ke) && eq(r1, r2)) == eq(r2, ke)
- where r1 and r2 are keys of elements in a_tran,
- n denotes a value of type size_type,
 - z denotes a value of type float, and
 - nh denotes a non-const rvalue of type X::node_type.

Table 4: Unordered associative container requirements
(in addition to container)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
X(i, j, n, hf, eq) X a(i, j, n, hf, eq);	X	<i>Preconditions:</i> value_type is Cpp17EmplaceConstructible into X from *i. <i>Effects:</i> Constructs an empty container with at least n buckets, using hf as the hash function and eq as the key equality predicate, and inserts elements from [i, j) into it.	Average case $\mathcal{O}(N)$ (N is distance(i, j)), worst case $\mathcal{O}(N^2)$
X(i, j, n, hf) X a(i, j, n, hf);	X	<i>Preconditions:</i> key_equal meets the Cpp17DefaultConstructible requirements. value_type is Cpp17EmplaceConstructible into X from *i. <i>Effects:</i> Constructs an empty container with at least n buckets, using hf as the hash function and key_equal() as the key equality predicate, and inserts elements from [i, j) into it.	Average case $\mathcal{O}(N)$ (N is distance(i, j)), worst case $\mathcal{O}(N^2)$

Table 4: Unordered associative container requirements
(in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>X(i, j, n)</code> <code>X a(i, j, n);</code>	X	<i>Preconditions:</i> hasher and key_equal meet the <i>Cpp17DefaultConstructible</i> requirements. value_type is <i>Cpp17EmplaceConstructible</i> into X from *i. <i>Effects:</i> Constructs an empty container with at least n buckets, using hasher() as the hash function and key_equal() as the key equality predicate, and inserts elements from [i, j) into it.	Average case $\mathcal{O}(N)$ (N is distance(i, j)), worst case $\mathcal{O}(N^2)$
<code>X(i, j)</code> <code>X a(i, j);</code>	X	<i>Preconditions:</i> hasher and key_equal meet the <i>Cpp17DefaultConstructible</i> requirements. value_type is <i>Cpp17EmplaceConstructible</i> into X from *i. <i>Effects:</i> Constructs an empty container with an unspecified number of buckets, using hasher() as the hash function and key_equal() as the key equality predicate, and inserts elements from [i, j) into it.	Average case $\mathcal{O}(N)$ (N is distance(i, j)), worst case $\mathcal{O}(N^2)$
<code>X(from_range, rg, n, hf, eq)</code> <code>X a(from_range, rg, n, hf, eq);</code>	X	<i>Preconditions:</i> value_type is <i>Cpp17EmplaceConstructible</i> into X from *ranges::begin(rg). <i>Effects:</i> Constructs an empty container with at least n buckets, using hf as the hash function and eq as the key equality predicate, and inserts each element from rg into it.	Average case $\mathcal{O}(N)$ (N is ranges::distance(rg)), worst case $\mathcal{O}(N^2)$

Table 4: Unordered associative container requirements
(in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>X(from_range, rg, n, hf)</code> <code>X a(from_range, rg, n, hf);</code>	X	<i>Preconditions:</i> key_equal meets the <i>Cpp17DefaultConstructible</i> requirements. value_type is <i>Cpp17EmplaceConstructible</i> into X from *ranges::begin(rg). <i>Effects:</i> Constructs an empty container with at least n buckets, using hf as the hash function and key_equal() as the key equality predicate, and inserts elements from rg into it.	Average case $\mathcal{O}(N)$ (N is ranges::distance(rg)), worst case $\mathcal{O}(N^2)$
<code>X(from_range, rg, n)</code> <code>X a(from_range, rg, n);</code>	X	<i>Preconditions:</i> hasher and key_equal meet the <i>Cpp17DefaultConstructible</i> requirements. value_type is <i>Cpp17EmplaceConstructible</i> into X from *ranges::begin(rg). <i>Effects:</i> Constructs an empty container with at least n buckets, using hasher() as the hash function and key_equal() as the key equality predicate, and inserts elements from rg into it.	Average case $\mathcal{O}(N)$ (N is ranges::distance(rg)), worst case $\mathcal{O}(N^2)$

Table 4: Unordered associative container requirements
(in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>X(from_range, rg)</code> <code>X a(from_range, rg);</code>	<code>X</code>	<i>Preconditions:</i> hasher and key_equal meet the <i>Cpp17DefaultConstructible</i> requirements. value_type is <i>Cpp17EmplaceConstructible</i> into X from <code>*ranges::begin(rg)</code> . <i>Effects:</i> Constructs an empty container with an unspecified number of buckets, using hasher() as the hash function and key_equal() as the key equality predicate, and inserts elements from rg into it.	Average case $\mathcal{O}(N)$ (N is <code>ranges::distance(rg)</code>), worst case $\mathcal{O}(N^2)$
<code>X(il)</code>	<code>X</code>	Same as <code>X(il.begin(), il.end())</code> .	Same as <code>X(il.begin(), il.end())</code> .
<code>a_uniq.insert(t)</code>	<code>pair<iterator, bool></code>	<i>Preconditions:</i> If t is a non-const rvalue, value_type is <i>Cpp17MoveInsertable</i> into X; otherwise, value_type is <i>Cpp17CopyInsertable</i> into X. <i>Effects:</i> Inserts t if and only if there is no element in the container with key equivalent to the key of t. The bool component of the returned pair indicates whether the insertion takes place, and the iterator component points to the element with key equivalent to the key of t.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a_uniq.size())$.

Table 4: Unordered associative container requirements
(in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>a_eq.insert(t)</code>	iterator	<i>Preconditions:</i> If <code>t</code> is a non-const rvalue, <code>value_type</code> is <i>Cpp17MoveInsertable</i> into <code>x</code> ; otherwise, <code>value_type</code> is <i>Cpp17CopyInsertable</i> into <code>x</code> . <i>Effects:</i> Inserts <code>t</code> , and returns an iterator pointing to the newly inserted element.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a_eq.size())$.
<code>a.insert(p, t)</code>	iterator	<i>Preconditions:</i> If <code>t</code> is a non-const rvalue, <code>value_type</code> is <i>Cpp17MoveInsertable</i> into <code>x</code> ; otherwise, <code>value_type</code> is <i>Cpp17CopyInsertable</i> into <code>x</code> . <i>Effects:</i> Equivalent to <code>a.insert(t)</code> . Return value is an iterator pointing to the element with the key equivalent to that of <code>t</code> . The iterator <code>p</code> is a hint pointing to where the search should start. Implementations are permitted to ignore the hint.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a.size())$.
<code>a.insert(i, j)</code>	void	<i>Preconditions:</i> <code>value_type</code> is <i>Cpp17EmplaceConstructible</i> into <code>x</code> from <code>*i</code> . Neither <code>i</code> nor <code>j</code> are iterators into <code>a</code> . <i>Effects:</i> Equivalent to <code>a.insert(t)</code> for each element <code>t</code> in <code>[i, j)</code> .	Average case $\mathcal{O}(N)$, where N is <code>distance(i, j)</code> , worst case $\mathcal{O}(N(a.size() + 1))$.

Table 4: Unordered associative container requirements
(in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
a.insert_range(rg)	void	<i>Preconditions:</i> <code>value_type</code> is <code>Cpp17EmplaceConstructible</code> into <code>x</code> from <code>*ranges::begin(rg)</code> . <code>rg</code> and <code>a</code> do not overlap. <i>Effects:</i> Equivalent to <code>a.insert(t)</code> for each element <code>t</code> in <code>rg</code> .	Average case $\mathcal{O}(N)$, where N is <code>ranges::distance(rg)</code> , worst case $\mathcal{O}(N(a.size() + 1))$.
<code>a.insert(il)</code>	<code>void</code>	Same as <code>a.insert(il.begin(), il.end())</code> .	Same as <code>a.insert(il.begin(), il.end())</code> .

Two unordered containers `a` and `b` compare equal if `a.size() == b.size()` and, for every equivalent-key group `[Ea1, Ea2)` obtained from `a.equal_range(Ea1)`, there exists an equivalent-key group `[Eb1, Eb2)` obtained from `b.equal_range(Ea1)`, such that `is_permutation(Ea1, Ea2, Eb1, Eb2)` returns true. For `unordered_set` and `unordered_map`, the complexity of `operator==` (i.e., the number of calls to the `==` operator of the `value_type`, to the predicate returned by `key_eq()`, and to the hasher returned by `hash_function()`) is proportional to N in the average case and to N^2 in the worst case, where N is `a.size()`. For `unordered_multiset` and `unordered_multimap`, the complexity of `operator==` is proportional to $\sum E_i^2$ in the average case and to N^2 in the worst case, where N is `a.size()`, and E_i is the size of the i^{th} equivalent-key group in `a`. However, if the respective elements of each corresponding pair of equivalent-key groups Ea_i and Eb_i are arranged in the same order (as is commonly the case, e.g., if `a` and `b` are unmodified copies of the same container), then the average-case complexity for `unordered_multiset` and `unordered_multimap` becomes proportional to N (but worst-case complexity remains $\mathcal{O}(N^2)$, e.g., for a pathologically bad hash function). The behavior of a program that uses `operator==` or `operator!=` on unordered containers is undefined unless the `Pred` function object has the same behavior for both containers and the equality comparison function for `Key` is a refinement¹ of the partition into equivalent-key groups produced by `Pred`.

The iterator types `iterator` and `const_iterator` of an unordered associative container are of at least the forward iterator category. For unordered associative containers where the key type and value type are the same, both `iterator` and `const_iterator` are constant iterators.

The `insert`, [insert_range](#), and `emplace` members shall not affect the validity of references to container elements, but may invalidate all iterators to the container. The `erase` members shall invalidate only iterators and references to the erased elements, and preserve the relative order of the elements that are not erased.

¹Equality comparison is a refinement of partitioning if no two objects that compare equal fall into different partitions.

The `insert`, [insert_range](#), and `emplace` members shall not affect the validity of iterators if $(N+n) \leq z * B$, where N is the number of elements in the container prior to the insert operation, n is the number of elements inserted, B is the container's bucket count, and z is the container's maximum load factor.

❖ Class template deque

[deque]

❖ Overview

[deque.overview]

```
namespace std {
    template<class T, class Allocator = allocator<T>>
    class deque {
    public:

        // ??, construct/copy/destroy
        deque() : deque(Allocator()) { }
        explicit deque(const Allocator&);
        explicit deque(size_type n, const Allocator& = Allocator());
        deque(size_type n, const T& value, const Allocator& = Allocator());
        template<class InputIterator>
        deque(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<container-compatible-range<T> R>
        deque\(from\_range\_t, R&& rg, const Allocator& = Allocator\(\)\);
        deque(const deque& x);
        deque(deque&&);
        deque(const deque&, const Allocator&);
        deque(deque&&, const Allocator&);
        deque(initializer_list<T>, const Allocator& = Allocator());

        ~deque();
        deque& operator=(const deque& x);
        deque& operator=(deque&& x);
        noexcept(allocator_traits<Allocator>::is_always_equal::value);
        deque& operator=(initializer_list<T>);
        template<class InputIterator>
        void assign(InputIterator first, InputIterator last);
        template<container-compatible-range<T> R>
        void assign\_range\(R&& rg\);
        void assign(size_type n, const T& t);
        void assign(initializer_list<T>);
        allocator_type get_allocator() const noexcept;
        //...

        void push_front(const T& x);
        void push_front(T&& x);
        template<container-compatible-range<T> R>
        void prepend\_range\(R&& rg\);
        void push_back(const T& x);
        void push_back(T&& x);
        template<container-compatible-range<T> R>
        void append\_range\(R&& rg\);
    };
}
```

```

        iterator insert(const_iterator position, const T& x);
        iterator insert(const_iterator position, T&& x);
        iterator insert(const_iterator position, size_type n, const T& x);
        template<class InputIterator>
        iterator insert(const_iterator position, InputIterator first, InputIterator last);
        template<container-compatible-range<T> R>
        iterator insert_range(const_iterator position, R&& rg);
        iterator insert(const_iterator position, initializer_list<T>);

        //...
};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
deque(InputIterator, InputIterator, Allocator = Allocator())
-> deque<iter-value-type<InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
deque(from_range_t, R&&, Allocator = Allocator())
-> deque<ranges::range_value_t<R>, Allocator>;
}

```

◆ Constructors, copy, and assignment [deque.cons]

```

template<class InputIterator>
deque(InputIterator first, InputIterator last, const Allocator& = Allocator());

```

Effects: Constructs a deque equal to the range [first, last), using the specified allocator.

Complexity: Linear in distance(first, last).

```

template<container-compatible-range<T> R>
deque(from_range_t, R&& rg, const Allocator& = Allocator());

```

Effects: Constructs a deque with the elements of the range rg, using the specified allocator.

Complexity: Linear in ranges::distance(rg).

◆ Modifiers [deque.modifiers]

```

iterator insert(const_iterator position, const T& x);
iterator insert(const_iterator position, T&& x);
iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
iterator insert(const_iterator position, InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
iterator insert_range(const_iterator position, R&& rg);
iterator insert(const_iterator position, initializer_list<T>);
template<class... Args> reference emplace_front(Args&&... args);
template<class... Args> reference emplace_back(Args&&... args);

```

```

template<class... Args> iterator emplace(const_iterator position, Args&&... args);
void push_front(const T& x);
void push_front(T&& x);
template<container-compatible-range<T> R>
void prepend\_range\(R&& rg\);
void push_back(const T& x);
void push_back(T&& x);
template<container-compatible-range<T> R>
void append\_range\(R&& rg\);

```

Effects: An insertion in the middle of the deque invalidates all the iterators and references to elements of the deque. An insertion at either end of the deque invalidates all the iterators to the deque, but has no effect on the validity of references to elements of the deque.

Complexity: The complexity is linear in the number of elements inserted plus the lesser of the distances to the beginning and end of the deque. Inserting a single element at either the beginning or end of a deque always takes constant time and causes a single call to a constructor of T.

Remarks: If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of T there are no effects. If an exception is thrown while inserting a single element at either end, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-*Cpp17CopyInsertable* T, the effects are unspecified.

❖ **Class template forward_list**

[forwardlist]

❖ **Overview**

[forwardlist.overview]

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class forward_list {
    public:

        // ??, construct/copy/destroy
        forward_list() : forward_list(Allocator()) { }
        explicit forward_list(const Allocator&);
        explicit forward_list(size_type n, const Allocator& = Allocator());
        forward_list(size_type n, const T& value, const Allocator& = Allocator());
        template<class InputIterator>
        forward_list(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<container-compatible-range<T> R>
        forward\_list\(from\_range\_t, R&& rg, const Allocator& = Allocator\(\)\);
        forward_list(const forward_list& x);
        forward_list(forward_list&& x);
        forward_list(const forward_list& x, const Allocator&);
        forward_list(forward_list&& x, const Allocator&);
        forward_list(initializer_list<T>, const Allocator& = Allocator());
        ~forward_list();
    };
}

```

```

forward_list& operator=(const forward_list& x);
forward_list& operator=(forward_list&& x)
noexcept(allocator_traits<Allocator>::is_always_equal::value);
forward_list& operator=(initializer_list<T>);
template<class InputIterator>
void assign(InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
void assign\_range\(R&& rg\);
void assign(size_type n, const T& t);
void assign(initializer_list<T>);
allocator_type get_allocator() const noexcept;

//...

template<class... Args> reference emplace_front(Args&&... args);
void push_front(const T& x);
void push_front(T&& x);
template<container-compatible-range<T> R>
void prepend\_range\(R&& rg\);
void pop_front();

template<class... Args> iterator emplace_after(const_iterator position, Args&&... args);
iterator insert_after(const_iterator position, const T& x);
iterator insert_after(const_iterator position, T&& x);

iterator insert_after(const_iterator position, size_type n, const T& x);
template<class InputIterator>
iterator insert_after(const_iterator position, InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
iterator insert\_range\_after\(const\_iterator position, R&& rg\);
iterator insert_after(const_iterator position, initializer_list<T> il);

};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
forward_list(InputIterator, InputIterator, Allocator = Allocator())
-> forward_list<iter-value-type<InputIterator>, Allocator>;

template<ranges::input\_range R, class Allocator = allocator<ranges::range\_value\_t<R>>>
forward\_list\(from\_range\_t, R&&, Allocator = Allocator\(\)\)
-> forward\_list<ranges::range\_value\_t<R>, Allocator>;
}

```



Constructors, copy, and assignment

[forwardlist.cons]

```

template<class InputIterator>
forward_list(InputIterator first, InputIterator last, const Allocator& = Allocator());

```

Effects: Constructs a forward_list object equal to the range [first, last).

Complexity: Linear in distance(first, last).

```
template<container-compatible-range<T> R>
forward_list(from_range_t, R&& rg, const Allocator& = Allocator());
```

Effects: Constructs a forward_list object with the elements of the range rg.

Complexity: Linear in `ranges::distance(rg)`.



Modifiers

[forward.list.modifiers]

```
void push_front(const T& x);
void push_front(T&& x);
```

Effects: Inserts a copy of x at the beginning of the list.

```
template<container-compatible-range<T> R>
void prepend_range(R&& rg);
```

Effects: Inserts a copy of each element of rg at the beginning of the list. [*Note:* The order of elements is not reversed. — end note]

...

```
template<class InputIterator>
iterator insert_after(const_iterator position, InputIterator first, InputIterator last);
```

Preconditions: position is before_begin() or is a dereferenceable iterator in the range [begin(), end()). Neither first nor last are iterators in *this.

Effects: Inserts copies of elements in [first, last) after position.

Returns: An iterator pointing to the last inserted element or position if first == last.

```
template<container-compatible-range<T> R>
iterator insert_range_after(const_iterator position, R&& rg);
```

Preconditions: position is before_begin() or is a dereferenceable iterator in the range [begin(), end()). rg and *this do not overlap.

Effects: Inserts copies of elements in the range rg after position.

Returns: An iterator pointing to the last inserted element or position if rg is empty.

```
iterator insert_after(const_iterator position, initializer_list<T> il);
```

Effects: insert_after(p, il.begin(), il.end()).

Returns: An iterator pointing to the last inserted element or position if il is empty.



Class template list

[list]



Overview

[list.overview]

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class list {
    public:

        // ??, construct/copy/destroy
        list() : list(Allocator()) { }
        explicit list(const Allocator&);
        explicit list(size_type n, const Allocator& = Allocator());
        list(size_type n, const T& value, const Allocator& = Allocator());
        template<class InputIterator>
        list(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<container-compatible-range<T> R>
        list\(from\_range\_t, R&& rg, const Allocator& = Allocator\(\)\);
        list(const list& x);
        list(list&& x);
        list(const list&, const Allocator&);
        list(list&&, const Allocator&);
        list(initializer_list<T>, const Allocator& = Allocator());
        ~list();
        list& operator=(const list& x);
        list& operator=(list&& x)
        noexcept(allocator_traits<Allocator>::is_always_equal::value);
        list& operator=(initializer_list<T>);
        template<class InputIterator>
        void assign(InputIterator first, InputIterator last);
        template<container-compatible-range<T> R>
        void assign\_range\(R&& rg\);
        void assign(size_type n, const T& t);
        void assign(initializer_list<T>);
        allocator_type get_allocator() const noexcept;

        //...

        void push_front(const T& x);
        void push_front(T&& x);
        template<container-compatible-range<T> R>
        void prepend\_range\(R&& rg\);
        void pop_front();
        void push_back(const T& x);
        void push_back(T&& x);
        template<container-compatible-range<T> R>
        void append\_range\(R&& rg\);
        void pop_back();

        template<class... Args> iterator emplace(const_iterator position, Args&&... args);
        iterator insert(const_iterator position, const T& x);
        iterator insert(const_iterator position, T&& x);
        iterator insert(const_iterator position, size_type n, const T& x);
        template<class InputIterator>
        iterator insert(const_iterator position, InputIterator first, InputIterator last);

```

```

    template<container-compatible-range<T> R>
    iterator insert_range(const_iterator position, R&& rg);
    iterator insert(const_iterator position, initializer_list<T> il);

    //...
};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
list(InputIterator, InputIterator, Allocator = Allocator())
-> list<iter-value-type<InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
list(from_range_t, R&&, Allocator = Allocator())
-> list<ranges::range_value_t<R>, Allocator>;
}

```

```

template<class InputIterator>
list(InputIterator first, InputIterator last, const Allocator& = Allocator());

```

Effects: Constructs a list equal to the range [first, last).

Complexity: Linear in distance(first, last).

```

template<container-compatible-range<T> R>
list(from_range_t, R&& rg, const Allocator& = Allocator());

```

Effects: Constructs a list object with the elements of the range rg.

Complexity: Linear in ranges::distance(rg).



Modifiers

[list.modifiers]

```

iterator insert(const_iterator position, const T& x);
iterator insert(const_iterator position, T&& x);
iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
iterator insert(const_iterator position, InputIterator first,
InputIterator last);
iterator insert(const_iterator position, initializer_list<T>);
template<container-compatible-range<T> R>
iterator insert_range(const_iterator position, R&& rg);

template<class... Args> reference emplace_front(Args&&... args);
template<class... Args> reference emplace_back(Args&&... args);
template<class... Args> iterator emplace(const_iterator position, Args&&... args);
void push_front(const T& x);
void push_front(T&& x);
template<container-compatible-range<T> R>
void prepend_range(R&& rg);
void push_back(const T& x);
void push_back(T&& x);

```

```
template<container-compatible-range<T> R>  
void append\_range\(R&& rg\);
```

Complexity: Insertion of a single element into a list takes constant time and exactly one call to a constructor of T. Insertion of multiple elements into a list is linear in the number of elements inserted, and the number of calls to the copy constructor or move constructor of T is exactly equal to the number of elements inserted.

Remarks: Does not affect the validity of iterators and references. If an exception is thrown there are no effects.

❖ **Class template vector** **[vector]**

❖ **Overview** **[vector.overview]**

A vector is a sequence container that supports (amortized) constant time insert and erase operations at the end; insert and erase in the middle take linear time. Storage management is handled automatically, though hints can be given to improve efficiency.

A vector meets all of the requirements of a container and of a reversible container (given in two tables in ??), of a sequence container, including most of the optional sequence container requirements, of an allocator-aware container (), and, for an element type other than bool, of a contiguous container. The exceptions are the push_front, [prepend_range](#), pop_front, and emplace_front member functions, which are not provided. Descriptions are provided here only for operations on vector that are not described in one of these tables or for operations where there is additional semantic information.

```
namespace std {  
    template<class T, class Allocator = allocator<T>>  
    class vector {  
    public:  
        // ??, construct/copy/destroy  
        constexpr vector() noexcept(noexcept(Allocator())) : vector(Allocator()) { }  
        constexpr explicit vector(const Allocator&) noexcept;  
        constexpr explicit vector(size_type n, const Allocator& = Allocator());  
        constexpr vector(size_type n, const T& value, const Allocator& = Allocator());  
        template<class InputIterator>  
        constexpr vector(InputIterator first, InputIterator last, const Allocator& = Allocator());  
        template<container-compatible-range<T> R>  
        constexpr vector\(from\_range\_t, R&& rg, const Allocator& = Allocator\(\)\);  
        constexpr vector(const vector& x);  
        constexpr vector(vector&&) noexcept;  
        constexpr vector(const vector&, const Allocator&);  
        constexpr vector(vector&&, const Allocator&);  
        constexpr vector(initializer_list<T>, const Allocator& = Allocator());  
        constexpr ~vector();  
        constexpr vector& operator=(const vector& x);  
        constexpr vector& operator=(vector&& x)  
        noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||  
        allocator_traits<Allocator>::is_always_equal::value);  
        constexpr vector& operator=(initializer_list<T>);  
    }  
};
```



```

template<class InputIterator>
constexpr void assign(InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
constexpr void assign\_range\(R&& rg\);
constexpr void assign(size_type n, const T& u);
constexpr void assign(initializer_list<T>);
constexpr allocator_type get_allocator() const noexcept;

//...

// ??, modifiers
template<class... Args> constexpr reference emplace_back(Args&&... args);
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
constexpr void append\_range\(R&& rg\);
constexpr void pop_back();

template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
constexpr iterator insert\_range\(const\_iterator position, R&& rg\);
constexpr iterator insert(const_iterator position, initializer_list<T> il);
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(vector&)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
allocator_traits<Allocator>::is_always_equal::value);
constexpr void clear() noexcept;
};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
vector(InputIterator, InputIterator, Allocator = Allocator())
-> vector<iter-value-type<InputIterator>, Allocator>;

template<ranges::input\_range R, class Allocator = allocator<ranges::range\_value\_t<R>>>
vector\(from\_range\_t, R&&, Allocator = Allocator\(\)\)
-> vector<ranges::range\_value\_t<R>, Allocator>;
}

```



Constructors

[vector.cons]

```

template<class InputIterator>
constexpr vector(InputIterator first, InputIterator last,
const Allocator& = Allocator());

```

Effects: Constructs a vector equal to the range `[first, last)`, using the specified allocator.

Complexity: Makes only N calls to the copy constructor of `T` (where N is the distance between `first` and `last`) and no reallocations if iterators `first` and `last` are of forward, bidirectional, or random access categories. It makes order N calls to the copy constructor of `T` and order $\log N$ reallocations if they are just input iterators.

```
template<container-compatible-range<T> R>
constexpr vector(from_range_t, R&& rg, const Allocator& = Allocator());
```

Effects: Constructs a vector object with the elements of the range `rg`, using the specified allocator.

Complexity: Initializes exactly N elements from the results of dereferencing successive iterators of `rg`, where N is `ranges::distance(rg)`. Performs no reallocations if `R` models `ranges::forward_range` or `ranges::sized_range`; otherwise, performs order $\log N$ reallocations and order N calls to the copy or move constructor of `T`.



Modifiers

[vector.modifiers]

```
constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T>);
template<class... Args> constexpr reference emplace_back(Args&&... args);
template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
constexpr void append_range(R&& rg);
```

Complexity: If reallocation happens, linear in the number of elements of the resulting vector; otherwise, linear in the number of elements inserted plus the distance to the end of the vector.

Remarks: Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. If no reallocation happens, then references, pointers, and iterators before the insertion point remain valid but those at or after the insertion point, including the past-the-end iterator, are invalidated. If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of `T` or by any `InputIterator` operation there are no effects. If an exception is thrown while inserting a single element at the end and `T` is `Cpp17CopyInsertable` or `is_nothrow_move_constructible_v<T>` is true, there are no effects. Otherwise,

if an exception is thrown by the move constructor of a non-*Cpp17CopyInsertable* *T*, the effects are unspecified.



Class `vector<bool>`

[`vector.bool`]

```
namespace std {
    template<class Allocator>
    class vector<bool, Allocator> {
    public:
        //...
        // construct/copy/destroy
        constexpr vector() : vector(Allocator()) { }
        constexpr explicit vector(const Allocator&);
        constexpr explicit vector(size_type n, const Allocator& = Allocator());
        constexpr vector(size_type n, const bool& value, const Allocator& = Allocator());
        template<class InputIterator>
        constexpr vector(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<container-compatible-range<bool> R>
        constexpr vector\(from\_range\_t, R&& rg, const Allocator& = Allocator\(\)\);
        constexpr vector(const vector& x);
        constexpr vector(vector&& x);
        constexpr vector(const vector&, const Allocator&);
        constexpr vector(vector&&, const Allocator&);
        constexpr vector(initializer_list<bool>, const Allocator& = Allocator());
        constexpr ~vector();
        constexpr vector& operator=(const vector& x);
        constexpr vector& operator=(vector&& x);
        constexpr vector& operator=(initializer_list<bool>);
        template<class InputIterator>
        constexpr void assign(InputIterator first, InputIterator last);
        template<container-compatible-range<bool> R>
        constexpr void assign\_range\(R&& rg\);
        constexpr void assign(size_type n, const bool& t);
        constexpr void assign(initializer_list<bool>);
        constexpr allocator_type get_allocator() const noexcept;
        //...

        // modifiers
        template<class... Args> constexpr reference emplace_back(Args&&... args);
        constexpr void push_back(const bool& x);
        template<container-compatible-range<bool> R>
        constexpr void append\_range\(R&& rg\);
        constexpr void pop_back();
        template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
        constexpr iterator insert(const_iterator position, const bool& x);
        constexpr iterator insert(const_iterator position, size_type n, const bool& x);
        template<class InputIterator>
        constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
        template<container-compatible-range<bool> R>
        constexpr iterator insert\_range\(const\_iterator position, R&& rg\);
        constexpr iterator insert(const_iterator position, initializer_list<bool> il);
    };
}
```

```
};
}
```

◆ Associative containers [associative]

◆ In general [associative.general]

The header `map` defines the class templates `map` and `multimap`; the header `set` defines the class templates `set` and `multiset`.

The following exposition-only alias templates may appear in deduction guides for associative containers:

```
template<class InputIterator>
using iter-value-type =
typename iterator_traits<InputIterator>::value_type;           // exposition only
template<class InputIterator>
using iter-key-type = remove_const_t<
typename iterator_traits<InputIterator>::value_type::first_type>; // exposition only
template<class InputIterator>
using iter-mapped-type =
typename iterator_traits<InputIterator>::value_type::second_type; // exposition only
template<class InputIterator>
using iter-to-alloc-type = pair<
add_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
typename iterator_traits<InputIterator>::value_type::second_type>; // exposition only

template<ranges::input_range Range>
using range-key-type =
    remove_const_t<typename ranges::range_value_t<Range>::first_type>; // exposition only

template<ranges::input_range Range>
using range-mapped-type = typename ranges::range_value_t<Range>::second_type; // exposition only

template<ranges::input_range Range>
using range-to-alloc-type =
    pair<add_const_t<typename ranges::range_value_t<Range>::first_type>,
        typename ranges::range_value_t<Range>::second_type>; // exposition only
```

◆ Class template `map` [map]

◆ Overview [map.overview]

```
namespace std {
template<class Key, class T, class Compare = less<Key>,
class Allocator = allocator<pair<const Key, T>>>
class map {
public:

    // ??, construct/copy/destroy
```

```

map() : map(Compare()) { }
explicit map(const Compare& comp, const Allocator& = Allocator());
template<class InputIterator>
map(InputIterator first, InputIterator last,
     const Compare& comp = Compare(), const Allocator& = Allocator());

template<container-compatible-range<value\_type> R>
map\(from\_range\_t, R&& rg, const Compare& comp = Compare\(\), const Allocator& = Allocator\(\)\);

map(const map& x);
map(map&& x);
explicit map(const Allocator&);
map(const map&, const Allocator&);
map(map&&, const Allocator&);
map(initializer_list<value_type>,
     const Compare& = Compare(),
     const Allocator& = Allocator());
template<class InputIterator>
map(InputIterator first, InputIterator last, const Allocator& a)
: map(first, last, Compare(), a) { }

template<container-compatible-range<value\_type> R>
map\(from\_range\_t, R&& rg, const Allocator& a\)
: map\(from\_range, std::forward<R>\(rg\), Compare\(\), a\){}

map(initializer_list<value_type> il, const Allocator& a)
: map(il, Compare(), a) { }

pair<iterator, bool> insert(const value_type& x);
pair<iterator, bool> insert(value_type&& x);
template<class P> pair<iterator, bool> insert(P&& x);
iterator insert(const_iterator position, const value_type& x);
iterator insert(const_iterator position, value_type&& x);
template<class P>
iterator insert(const_iterator position, P&&);
template<class InputIterator>
void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value\_type> R>
void insert\_range\(R&& rg\);

void insert(initializer_list<value_type>);

//...
};

template<class InputIterator, class Compare = less<iter-key-type<InputIterator>>,
class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
map(InputIterator, InputIterator, Compare = Compare(), Allocator = Allocator())
-> map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Compare, Allocator>;

template<ranges::input\_range R,
class Compare = less<range-key-type<R>,

```

```

class Allocator = allocator<range-to-alloc-type<R>>>
map(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> map<range-key-type<R>, range-mapped-type<R>, Compare, Allocator>;

```

```

template<class Key, class T, class Compare = less<Key>,
class Allocator = allocator<pair<const Key, T>>>
map(initializer_list<pair<Key, T>>, Compare = Compare(), Allocator = Allocator())
-> map<Key, T, Compare, Allocator>;

```

```

template<class InputIterator, class Allocator>
map(InputIterator, InputIterator, Allocator)
-> map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
less<iter-key-type<InputIterator>>, Allocator>;

```

```

template<ranges::input_range R, class Allocator>
map(from_range_t, R&&, Allocator)
-> map<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>, Allocator>;

```

```

template<class Key, class T, class Allocator>
map(initializer_list<pair<Key, T>>, Allocator) -> map<Key, T, less<Key>, Allocator>;
}

```

❖ Constructors, copy, and assignment

[map.cons]

```

template<class InputIterator>
map(InputIterator first, InputIterator last,
const Compare& comp = Compare(), const Allocator& = Allocator());

```

Effects: Constructs an empty map using the specified comparison object and allocator, and inserts elements from the range [first, last).

Complexity: Linear in N if the range [first, last) is already sorted using comp and otherwise $N \log N$, where N is last - first.

```

template<container-compatible-range<value_type> R>
map(from_range_t, R&& rg, const Compare& comp = Compare(), const Allocator& = Allocator());

```

Effects: Constructs an empty map using the specified comparison object and allocator, and inserts elements from the range rg.

Complexity: Linear in N if rg is already sorted using comp and otherwise $N \log N$, where N is ranges::distance(rg).

.

❖ Class template multimap

[multimap]

❖ Overview

[multimap.overview]

```

namespace std {
template<class Key, class T, class Compare = less<Key>,
class Allocator = allocator<pair<const Key, T>>>
class multimap {

    // ??, construct/copy/destroy
    multimap() : multimap(Compare()) { }
    explicit multimap(const Compare& comp, const Allocator& = Allocator());
    template<class InputIterator>
    multimap(InputIterator first, InputIterator last, const Compare& comp = Compare(), const Allocator& = Allocator());
    template<container-compatible-range<value\_type> R>
    multimap\(from\_range\_t, R&& rg, const Compare& comp = Compare\(\), const Allocator& = Allocator\(\)\);
    multimap(const multimap& x);
    multimap(multimap&& x);
    explicit multimap(const Allocator&);
    multimap(const multimap&, const Allocator&);
    multimap(multimap&&, const Allocator&);
    multimap(initializer_list<value_type>,
    const Compare& = Compare(),
    const Allocator& = Allocator());
    template<class InputIterator>
    multimap(InputIterator first, InputIterator last, const Allocator& a)
    : multimap(first, last, Compare(), a) { }

    template<container-compatible-range<value\_type> R>
    multimap\(from\_range\_t, R&& rg, const Allocator& a\)
    : multimap\(from\_range, std::forward<R>\(rg\), Compare\(\), a\){}

    multimap(initializer_list<value_type> il, const Allocator& a)
    : multimap(il, Compare(), a) { }
    ~multimap();

    // ??, modifiers
    template<class... Args> iterator emplace(Args&&... args);
    template<class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
    iterator insert(const value_type& x);
    iterator insert(value_type&& x);
    template<class P> iterator insert(P&& x);
    iterator insert(const_iterator position, const value_type& x);
    iterator insert(const_iterator position, value_type&& x);
    template<class P> iterator insert(const_iterator position, P&& x);
    template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
    template<container-compatible-range<value\_type> R>
    void insert\_range\(R&& rg\);
    void insert(initializer_list<value_type>);
};

template<class InputIterator, class Compare = less<iter-key-type<InputIterator>>,
class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
multimap(InputIterator, InputIterator, Compare = Compare(), Allocator = Allocator())

```

```

-> multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
Compare, Allocator>;

template<ranges::input_range R,
        class Compare = less<range-key-type<R>>,
        class Allocator = allocator<range-to-alloc-type<R>>>
multimap(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> multimap<range-key-type<R>, range-mapped-type<R>, Compare, Allocator>;

template<class Key, class T, class Compare = less<Key>,
class Allocator = allocator<pair<const Key, T>>>
multimap(initializer_list<pair<Key, T>>, Compare = Compare(), Allocator = Allocator())
-> multimap<Key, T, Compare, Allocator>;

template<class InputIterator, class Allocator>
multimap(InputIterator, InputIterator, Allocator)
-> multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
less<iter-key-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
multimap(from_range_t, R&&, Allocator)
-> multimap<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>, Allocator>;

template<class Key, class T, class Allocator>
multimap(initializer_list<pair<Key, T>>, Allocator)
-> multimap<Key, T, less<Key>, Allocator>;
}

```



Constructors

[multimap.cons]

```

template<class InputIterator>
multimap(InputIterator first, InputIterator last,
const Compare& comp = Compare(),
const Allocator& = Allocator());

```

Effects: Constructs an empty multimap using the specified comparison object and allocator, and inserts elements from the range [first, last).

Complexity: Linear in N if the range [first, last) is already sorted using comp and otherwise $N \log N$, where N is last - first.

```

template<container-compatible-range<value_type> R>
multimap(from_range_t, R&& rg, const Compare& comp = Compare(), const Allocator& = Allocator());

```

Effects: Constructs an empty multimap using the specified comparison object and allocator, and inserts elements from the range rg.

Complexity: Linear in N if rg is already sorted using comp and otherwise $N \log N$, where N is ranges::distance(rg).

.



Class template set

[set]



Overview

[set.overview]

```

namespace std {
    template<class Key, class Compare = less<Key>,
            class Allocator = allocator<Key>>
    class set {
    public:
        // ??, construct/copy/destroy
        set() : set(Compare()) { }
        explicit set(const Compare& comp, const Allocator& = Allocator());
        template<class InputIterator>
        set(InputIterator first, InputIterator last, const Compare& comp = Compare(), const Allocator& = Allocator());
        template<container-compatible-range<value\_type> R>
        set\(from\_range\_t, R&& rg, const Compare& comp = Compare\(\), const Allocator& = Allocator\(\)\);

        set(const set& x);
        set(set&& x);
        explicit set(const Allocator&);
        set(const set&, const Allocator&);
        set(set&&, const Allocator&);
        set(initializer_list<value_type>, const Compare& = Compare(),
            const Allocator& = Allocator());
        template<class InputIterator>
        set(InputIterator first, InputIterator last, const Allocator& a)
        : set(first, last, Compare(), a) { }
        template<container-compatible-range<value\_type> R>
        set\(from\_range\_t, R&& rg, const Allocator& a\)
        : set\(from\_range, std::forward<R>\(rg\), Compare\(\), a\){}

        set(initializer_list<value_type> il, const Allocator& a)
        : set(il, Compare(), a) { }
        ~set();
        //...
        pair<iterator,bool> insert(const value_type& x);
        pair<iterator,bool> insert(value_type&& x);
        iterator insert(const_iterator position, const value_type& x);
        iterator insert(const_iterator position, value_type&& x);
        template<class InputIterator>
        void insert(InputIterator first, InputIterator last);

        template<container-compatible-range<value\_type> R>
        void insert\_range\(R&& rg\);

        void insert(initializer_list<value_type>);

    };

    template<class InputIterator,
            class Compare = less<iter-value-type<InputIterator>>,

```

```

class Allocator = allocator<iter-value-type<InputIterator>>>
set(InputIterator, InputIterator,
    Compare = Compare(), Allocator = Allocator())
-> set<iter-value-type<InputIterator>, Compare, Allocator>;

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
    class Allocator = allocator<ranges::range_value_t<R>>>
set(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> set<ranges::range_value_t<R>, Compare, Allocator>;

template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
set(initializer_list<Key>, Compare = Compare(), Allocator = Allocator())
-> set<Key, Compare, Allocator>;

template<class InputIterator, class Allocator>
set(InputIterator, InputIterator, Allocator)
-> set<iter-value-type<InputIterator>,
    less<iter-value-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
set(from_range_t, R&&, Allocator)
-> set<ranges::range_value_t<R>, less<ranges::range_value_t<R>>, Allocator>;

template<class Key, class Allocator>
set(initializer_list<Key>, Allocator) -> set<Key, less<Key>, Allocator>;
}

```



Constructors, copy, and assignment

[set.cons]

```

template<class InputIterator>
set(InputIterator first, InputIterator last,
    const Compare& comp = Compare(), const Allocator& = Allocator());

```

Effects: Constructs an empty set using the specified comparison object and allocator, and inserts elements from the range [first, last).

Complexity: Linear in N if the range [first, last) is already sorted using comp and otherwise $N \log N$, where N is last - first.

```

template<container-compatible-range<value_type> R>
set(from_range_t, R&& rg, const Compare& comp = Compare(), const Allocator& = Allocator());

```

Effects: Constructs an empty set using the specified comparison object and allocator, and inserts elements from the range rg.

Complexity: Linear in N if rg is already sorted using comp and otherwise $N \log N$, where N is ranges::distance(rg).

❖ Class template multiset

[multiset]

❖ Overview

[multiset.overview]

```
namespace std {
template<class Key, class Compare = less<Key>,
class Allocator = allocator<Key>>
class multiset {
public:
    // ??, construct/copy/destroy
    multiset() : multiset(Compare()) { }
    explicit multiset(const Compare& comp, const Allocator& = Allocator());
    template<class InputIterator>
    multiset(InputIterator first, InputIterator last, const Compare& comp = Compare(),
             const Allocator& = Allocator());

    template<container-compatible-range<value\_type> R>
    multiset\(from\_range\_t, R&& rg, const Compare& comp = Compare\(\), const Allocator& = Allocator\(\)\);

    multiset(const multiset& x);
    multiset(multiset&& x);
    explicit multiset(const Allocator&);
    multiset(const multiset&, const Allocator&);
    multiset(multiset&&, const Allocator&);
    multiset(initializer_list<value_type>, const Compare& = Compare(),
             const Allocator& = Allocator());
    template<class InputIterator>
    multiset(InputIterator first, InputIterator last, const Allocator& a)
    : multiset(first, last, Compare(), a) { }

    template<container-compatible-range<value\_type> R>
    multiset\(from\_range\_t, R&& rg, const Allocator& a\)
    : multiset\(from\_range, std::forward<R>\(rg\), Compare\(\), a\){}

    multiset(initializer_list<value_type> il, const Allocator& a)
    : multiset(il, Compare(), a) { }
    ~multiset();

    iterator insert(const value_type& x);
    iterator insert(value_type&& x);
    iterator insert(const_iterator position, const value_type& x);
    iterator insert(const_iterator position, value_type&& x);
    template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
    template<container-compatible-range<value\_type> R>
    void insert\_range\(R&& rg\);

    void insert(initializer_list<value_type>);

};

template<class InputIterator,
```

```

class Compare = less<iter-value-type<InputIterator>>,
class Allocator = allocator<iter-value-type<InputIterator>>>
multiset(InputIterator, InputIterator,
Compare = Compare(), Allocator = Allocator())
-> multiset<iter-value-type<InputIterator>, Compare, Allocator>;

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
      class Allocator = allocator<ranges::range_value_t<R>>>
multiset(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> multiset<ranges::range_value_t<R>, Compare, Allocator>;

template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
multiset(initializer_list<Key>, Compare = Compare(), Allocator = Allocator())
-> multiset<Key, Compare, Allocator>;

template<class InputIterator, class Allocator>
multiset(InputIterator, InputIterator, Allocator)
-> multiset<iter-value-type<InputIterator>,
      less<iter-value-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
multiset(from_range_t, R&&, Allocator)
-> multiset<ranges::range_value_t<R>, less<ranges::range_value_t<R>>, Allocator>;

template<class Key, class Allocator>
multiset(initializer_list<Key>, Allocator) -> multiset<Key, less<Key>, Allocator>;
}

```



Constructors

[multiset.cons]

```

template<class InputIterator>
multiset(InputIterator first, InputIterator last,
const Compare& comp = Compare(), const Allocator& = Allocator());

```

Effects: Constructs an empty multiset using the specified comparison object and allocator, and inserts elements from the range [first, last).

Complexity: Linear in N if the range [first, last) is already sorted using comp and otherwise $N \log N$, where N is last - first.

```

template<container-compatible-range<value_type> R>
set(from_range_t, R&& rg, const Compare& comp = Compare(), const Allocator& = Allocator());

```

Effects: Constructs an empty multiset using the specified comparison object and allocator, and inserts elements from the range rg.

Complexity: Linear in N if rg is already sorted using comp and otherwise $N \log N$, where N is ranges::distance(rg).

.



Unordered associative containers

[unord]



Class template unordered_map

[unord.map]



Overview

[unord.map.overview]

```

namespace std {
template<class Key,
class T,
class Hash = hash<Key>,
class Pred = equal_to<Key>,
class Allocator = allocator<pair<const Key, T>>>
class unordered_map {
    // ??, construct/copy/destroy
    unordered_map();
    explicit unordered_map(size_type n, const hasher& hf = hasher(),
        const key_equal& eql = key_equal(), const allocator_type& a = allocator_type());
    template<class InputIterator>
    unordered_map(InputIterator f, InputIterator l, size_type n = see below,
        const hasher& hf = hasher(), const key_equal& eql = key_equal(),
        const allocator_type& a = allocator_type());

    template<container-compatible-range<value\_type> R>
    unordered\_map\(from\_range\_t, R&& rg, size\_type n = see below,
    const hasher& hf = hasher\(\), const key\_equal& eql = key\_equal\(\),
    const allocator\_type& a = allocator\_type\(\)\);

    unordered_map(const unordered_map&);
    unordered_map(unordered_map&&);
    explicit unordered_map(const Allocator&);
    unordered_map(const unordered_map&, const Allocator&);
    unordered_map(unordered_map&&, const Allocator&);
    unordered_map(initializer_list<value_type> il, size_type n = see below,
        const hasher& hf = hasher(),
        const key_equal& eql = key_equal(),
        const allocator_type& a = allocator_type());
    unordered_map(size_type n, const allocator_type& a)
    : unordered_map(n, hasher(), key_equal(), a) { }
    unordered_map(size_type n, const hasher& hf, const allocator_type& a)
    : unordered_map(n, hf, key_equal(), a) { }

    template<class InputIterator>
    unordered_map(InputIterator f, InputIterator l, size_type n, const allocator_type& a)
    : unordered_map(f, l, n, hasher(), key_equal(), a) { }
    template<class InputIterator>
    unordered_map(InputIterator f, InputIterator l, size_type n, const hasher& hf,
        const allocator_type& a)
    : unordered_map(f, l, n, hf, key_equal(), a) { }

    template<container-compatible-range<value\_type> R>
    unordered\_map\(from\_range\_t, R&& rg, size\_type n, const allocator\_type& a\)

```

```

: unordered_map(from_range, std::forward<R>(rg), n, hasher(), key_equal(), a) {}

template<container-compatible-range<value_type> R>
unordered_map(from_range_t, R&& rg, size_type n, const hasher& hf, const allocator_type& a)
: unordered_map(from_range, std::forward<R>(rg), n, hf, key_equal(), a) {}

unordered_map(initializer_list<value_type> il, size_type n, const allocator_type& a)
: unordered_map(il, n, hasher(), key_equal(), a) { }
unordered_map(initializer_list<value_type> il, size_type n, const hasher& hf,
const allocator_type& a)
: unordered_map(il, n, hf, key_equal(), a) { }
~unordered_map();

// ??, modifiers
template<class... Args> pair<iterator, bool> emplace(Args&&... args);
template<class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
pair<iterator, bool> insert(const value_type& obj);
pair<iterator, bool> insert(value_type&& obj);
template<class P> pair<iterator, bool> insert(P&& obj);
iterator      insert(const_iterator hint, const value_type& obj);
iterator      insert(const_iterator hint, value_type&& obj);
template<class P> iterator insert(const_iterator hint, P&& obj);
template<class InputIterator> void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
void insert_range(R&& rg);
void insert(initializer_list<value_type>);

};

template<class InputIterator,
class Hash = hash<iter-key-type<InputIterator>>,
class Pred = equal_to<iter-key-type<InputIterator>>,
class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
unordered_map(InputIterator, InputIterator, typename see below::size_type = see below,
Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash, Pred,
Allocator>;

template<ranges::input_range R,
class Hash = hash<range-key-type<R>>,
class Pred = equal_to<range-key-type<R>>,
class Allocator = allocator<range-to-alloc-type<R>>>
unordered_map(from_range_t, R&&, typename see below::size_type = see below,
Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_map<range-key-type<R>, range-mapped-type<R>,
Hash, Pred, Allocator>;

template<class Key, class T, class Hash = hash<Key>,
class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>

```

```

unordered_map(initializer_list<pair<Key, T>>,
typename see below::size_type = see below, Hash = Hash(),
Pred = Pred(), Allocator = Allocator())
-> unordered_map<Key, T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_map(InputIterator, InputIterator, typename see below::size_type, Allocator)
-> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
hash<iter-key-type<InputIterator>>,
equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Allocator>
unordered_map(InputIterator, InputIterator, Allocator)
-> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
hash<iter-key-type<InputIterator>>,
equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_map(InputIterator, InputIterator, typename see below::size_type, Hash, Allocator)
-> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash,
equal_to<iter-key-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
unordered_map(from_range_t, R&&, typename see below::size_type, Allocator)
-> unordered_map<range-key-type<R>,
    range-mapped-type<R>, hash<range-key-type<R>>,
    equal_to<range-key-type<R>>, Allocator>;

template<ranges::input_range R, class Allocator>
unordered_map(from_range_t, R&&, Allocator)
-> unordered_map<range-key-type<R>,
    range-mapped-type<R>, hash<range-key-type<R>>, equal_to<range-key-type<R>>, Allocator>;

template<ranges::input_range R, class Hash, class Allocator>
unordered_map(from_range_t, R&&, typename see below::size_type, Hash, Allocator)
-> unordered_map<range-key-type<R>, range-mapped-type<R>, Hash, equal_to<range-key-type<R>>, Allocator>;

template<class Key, class T, class Allocator>
unordered_map(initializer_list<pair<Key, T>>, typename see below::size_type,
Allocator)
-> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Allocator>
unordered_map(initializer_list<pair<Key, T>>, Allocator)
-> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
unordered_map(initializer_list<pair<Key, T>>, typename see below::size_type, Hash,
Allocator)
-> unordered_map<Key, T, Hash, equal_to<Key>, Allocator>;

```

```
}
```

❖ Constructors

[unord.map.cnstr]

```
template<class InputIterator>
unordered_map(InputIterator f, InputIterator l,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
template<container-compatible-range<value_type> R>
unordered_map(from_range_t, R&& rg, size_type n = see below,
    const hasher& hf = hasher(), const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
unordered_map(initializer_list<value_type> il,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
```

Effects: Constructs an empty unordered_map using the specified hash function, key equality predicate, and allocator, and using at least n buckets. If n is not provided, the number of buckets is implementation-defined. Then inserts elements from the range [f, l) **for the first form, or from the range [il.begin(), il.end()) for the second form**, rg, or il, respectively. max_load_factor() returns 1.0.

Complexity: Average case linear, worst case quadratic.

❖ Class template unordered_multimap

[unord.multimap]

❖ Overview

[unord.multimap.overview]

```
namespace std {
template<class Key,
class T,
class Hash = hash<Key>,
class Pred = equal_to<Key>,
class Allocator = allocator<pair<const Key, T>>>
class unordered_multimap {
public:
    // ??, construct/copy/destroy
    unordered_multimap();
    explicit unordered_multimap(size_type n,
        const hasher& hf = hasher(),
        const key_equal& eql = key_equal(),
        const allocator_type& a = allocator_type());
    template<class InputIterator>
    unordered_multimap(InputIterator f, InputIterator l,
        size_type n = see below,
        const hasher& hf = hasher(),
```



```
const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());
```

```
template<container-compatible-range<value_type> R>
unordered_multimap(from_range_t, R&& rg, size_type n = see below,
const hasher& hf = hasher(), const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());
```

```
unordered_multimap(const unordered_multimap&);
unordered_multimap(unordered_multimap&&);
explicit unordered_multimap(const Allocator&);
unordered_multimap(const unordered_multimap&, const Allocator&);
unordered_multimap(unordered_multimap&&, const Allocator&);
unordered_multimap(initializer_list<value_type> il,
size_type n = see below,
const hasher& hf = hasher(),
const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());
unordered_multimap(size_type n, const allocator_type& a)
: unordered_multimap(n, hasher(), key_equal(), a) { }
unordered_multimap(size_type n, const hasher& hf, const allocator_type& a)
: unordered_multimap(n, hf, key_equal(), a) { }
template<class InputIterator>
unordered_multimap(InputIterator f, InputIterator l, size_type n, const allocator_type& a)
: unordered_multimap(f, l, n, hasher(), key_equal(), a) { }
template<class InputIterator>
unordered_multimap(InputIterator f, InputIterator l, size_type n, const hasher& hf,
const allocator_type& a)
: unordered_multimap(f, l, n, hf, key_equal(), a) { }
```

```
template<container-compatible-range<value_type> R>
unordered_multimap(from_range_t, R&& rg, size_type n, const allocator_type& a)
: unordered_multimap(from_range, std::forward<R>(rg), n, hasher(), key_equal(), a) {}
```

```
template<container-compatible-range<value_type> R>
unordered_multimap(from_range_t, R&& rg, size_type n, const hasher& hf, const allocator_type&
```

a)

```
: unordered_multimap(from_range, std::forward<R>(rg), n, hf, key_equal(), a) {}
```

```
unordered_multimap(initializer_list<value_type> il, size_type n, const allocator_type& a)
: unordered_multimap(il, n, hasher(), key_equal(), a) { }
unordered_multimap(initializer_list<value_type> il, size_type n, const hasher& hf,
const allocator_type& a)
: unordered_multimap(il, n, hf, key_equal(), a) { }
```

```
// ??, modifiers
template<class... Args> iterator emplace(Args&&... args);
template<class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
iterator insert(const value_type& obj);
iterator insert(value_type&& obj);
```

```

    template<class P> iterator insert(P&& obj);
    iterator insert(const_iterator hint, const value_type& obj);
    iterator insert(const_iterator hint, value_type&& obj);
    template<class P> iterator insert(const_iterator hint, P&& obj);
    template<class InputIterator> void insert(InputIterator first, InputIterator last);
    template<container-compatible-range<value_type> R>
    void insert_range(R&& rg);
    void insert(initializer_list<value_type>);

};

template<class InputIterator,
class Hash = hash<iter-key-type<InputIterator>>,
class Pred = equal_to<iter-key-type<InputIterator>>,
class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
unordered_multimap(InputIterator, InputIterator,
typename see below::size_type = see below,
Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
Hash, Pred, Allocator>;

template<ranges::input_range R,
class Hash = hash<range-key-type<R>>,
class Pred = equal_to<range-key-type<R>>,
class Allocator = allocator<range-to-alloc-type<R>>>
unordered_multimap(from_range_t, R&&, typename see below::size_type = see below,
Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<range-key-type<R>,
range-mapped-type<R>,
Hash, Pred, Allocator>;

template<class Key, class T, class Hash = hash<Key>,
class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_multimap(initializer_list<pair<Key, T>>,
typename see below::size_type = see below,
Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<Key, T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, typename see below::size_type, Allocator)
-> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
hash<iter-key-type<InputIterator>>,
equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, Allocator)
-> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
hash<iter-key-type<InputIterator>>,
equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>

```

```
unordered_multimap(InputIterator, InputIterator, typename see below::size_type, Hash,
Allocator)
-> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash,
equal_to<iter-key-type<InputIterator>>, Allocator>;
```

```
template<ranges::input_range R, class Allocator>
unordered_multimap(from_range_t, R&&, typename see below::size_type, Allocator)
-> unordered_multimap<range-key-type<R>,
    range-mapped-type<R>,
    hash<range-key-type<R>>,
    equal_to<range-key-type<R>>, Allocator>;
```

```
template<ranges::input_range R, class Allocator>
unordered_multimap(from_range_t, R&&, Allocator)
-> unordered_multimap<range-key-type<R>,
    range-mapped-type<R>,
    hash<range-key-type<R>>,
    equal_to<range-key-type<R>>, Allocator>;
```

```
template<ranges::input_range R, class Hash, class Allocator>
unordered_multimap(from_range_t, R&&, typename see below::size_type, Hash, Allocator)
-> unordered_multimap<range-key-type<R>,
    range-mapped-type<R>,
    Hash,
    equal_to<range-key-type<R>>, Allocator>;
```

```
template<class Key, class T, class Allocator>
unordered_multimap(initializer_list<pair<Key, T>>, typename see below::size_type,
Allocator)
-> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;
```

```
template<class Key, class T, class Allocator>
unordered_multimap(initializer_list<pair<Key, T>>, Allocator)
-> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;
```

```
template<class Key, class T, class Hash, class Allocator>
unordered_multimap(initializer_list<pair<Key, T>>, typename see below::size_type,
Hash, Allocator)
-> unordered_multimap<Key, T, Hash, equal_to<Key>, Allocator>;
}
```

A `size_type` parameter type in an `unordered_multimap` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

◆ Constructors

[unord.multimap.cnstr]

```
template<class InputIterator>
unordered_multimap(InputIterator f, InputIterator l,
    size_type n = see below,
    const hasher& hf = hasher(),
```

```

const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());
template<container-compatible-range<value_type> R>
unordered_multimap(from_range_t, R&& rg, size_type n = see below,
const hasher& hf = hasher(), const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());
unordered_multimap(initializer_list<value_type> il,
size_type n = see below,
const hasher& hf = hasher(),
const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());

```

Effects: Constructs an empty unordered_multimap using the specified hash function, key equality predicate, and allocator, and using at least n buckets. If n is not provided, the number of buckets is implementation-defined. Then inserts elements from the range [f, 1) **for the first form, or from the range [il.begin(), il.end()) for the second form, rg, or il, respectively.** max_load_factor() returns 1.0.

Complexity: Average case linear, worst case quadratic.

❖ **Class template unordered_set**

[unord.set]

❖ **Overview**

[unord.set.overview]

```

namespace std {
template<class Key,
class Hash = hash<Key>,
class Pred = equal_to<Key>,
class Allocator = allocator<Key>>
class unordered_set {
public:

// ??, construct/copy/destroy
unordered_set();
explicit unordered_set(size_type n,
const hasher& hf = hasher(),
const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());
template<class InputIterator>
unordered_set(InputIterator f, InputIterator l,
size_type n = see below,
const hasher& hf = hasher(),
const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());

template<container-compatible-range<value_type> R>
unordered_set(from_range_t, R&& rg, size_type n = see below,
const hasher& hf = hasher(), const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());

unordered_set(const unordered_set&);

```

```

unordered_set(unordered_set&&);
explicit unordered_set(const Allocator&);
unordered_set(const unordered_set&, const Allocator&);
unordered_set(unordered_set&&, const Allocator&);
unordered_set(initializer_list<value_type> il,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
unordered_set(size_type n, const allocator_type& a)
: unordered_set(n, hasher(), key_equal(), a) { }
unordered_set(size_type n, const hasher& hf, const allocator_type& a)
: unordered_set(n, hf, key_equal(), a) { }
template<class InputIterator>
unordered_set(InputIterator f, InputIterator l, size_type n, const allocator_type& a)
: unordered_set(f, l, n, hasher(), key_equal(), a) { }
template<class InputIterator>
unordered_set(InputIterator f, InputIterator l, size_type n, const hasher& hf,
    const allocator_type& a)
: unordered_set(f, l, n, hf, key_equal(), a) { }

template<container-compatible-range<value\_type> R>
unordered\_set\(from\_range\_t, R&& rg, size\_type n, const allocator\_type& a\)
: unordered\_set\(from\_range, std::forward<R>\(rg\), n, hasher\(\), key\_equal\(\), a\) {}

template<container-compatible-range<value\_type> R>
unordered\_set\(from\_range\_t, R&& rg, size\_type n, const hasher& hf, const allocator\_type& a\)
: unordered\_set\(from\_range, std::forward<R>\(rg\), n, hf, key\_equal\(\), a\) {}

unordered_set(initializer_list<value_type> il, size_type n, const allocator_type& a)
: unordered_set(il, n, hasher(), key_equal(), a) { }
unordered_set(initializer_list<value_type> il, size_type n, const hasher& hf,
    const allocator_type& a)
: unordered_set(il, n, hf, key_equal(), a) { }

// modifiers
template<class... Args> pair<iterator, bool> emplace(Args&&... args);
template<class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
pair<iterator, bool> insert(const value_type& obj);
pair<iterator, bool> insert(value_type&& obj);
iterator insert(const_iterator hint, const value_type& obj);
iterator insert(const_iterator hint, value_type&& obj);
template<class InputIterator> void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value\_type> R>
void insert\_range\(R&& rg\);
void insert(initializer_list<value_type>);

};

template<class InputIterator,
    class Hash = hash<iter-value-type<InputIterator>>,

```

```

    class Pred = equal_to<iter-value-type<InputIterator>>,
    class Allocator = allocator<iter-value-type<InputIterator>>>
unordered_set(InputIterator, InputIterator, typename see below::size_type = see below,
    Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_set<iter-value-type<InputIterator>,
    Hash, Pred, Allocator>;

template<ranges::input_range R,
    class Hash = hash<ranges::range_value_t<R>>,
    class Pred = equal_to<ranges::range_value_t<R>>,
    class Allocator = allocator<ranges::range_value_t<R>>>
unordered_set(from_range_t, R&&, typename see below::size_type = see below,
    Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_set<ranges::range_value_t<R>, Hash, Pred, Allocator>;

template<class T, class Hash = hash<T>,
    class Pred = equal_to<T>, class Allocator = allocator<T>>
unordered_set(initializer_list<T>, typename see below::size_type = see below,
    Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_set<T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_set(InputIterator, InputIterator, typename see below::size_type, Allocator)
-> unordered_set<iter-value-type<InputIterator>,
    hash<iter-value-type<InputIterator>>,
    equal_to<iter-value-type<InputIterator>>,
    Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_set(InputIterator, InputIterator, typename see below::size_type,
    Hash, Allocator)
-> unordered_set<iter-value-type<InputIterator>, Hash,
    equal_to<iter-value-type<InputIterator>>,
    Allocator>;

template<ranges::input_range R, class Allocator>
unordered_set(from_range_t, R&&, typename see below::size_type, Allocator)
-> unordered_set<ranges::range_value_t<R>,
    hash<ranges::range_value_t<R>>,
    equal_to<ranges::range_value_t<R>>, Allocator>;

template<ranges::input_range R, class Allocator>
unordered_set(from_range_t, R&&, Allocator)
-> unordered_set<ranges::range_value_t<R>,
    hash<ranges::range_value_t<R>>,
    equal_to<ranges::range_value_t<R>>, Allocator>;

template<ranges::input_range R, class Hash, class Allocator>
unordered_set(from_range_t, R&&, typename see below::size_type, Hash, Allocator)
-> unordered_set<ranges::range_value_t<R>,
    Hash,

```

```
equal_to<ranges::range_value_t<R>>, Allocator>;
```

```
template<class T, class Allocator>
unordered_set(initializer_list<T>, typename see below::size_type, Allocator)
-> unordered_set<T, hash<T>, equal_to<T>, Allocator>;

template<class T, class Hash, class Allocator>
unordered_set(initializer_list<T>, typename see below::size_type, Hash, Allocator)
-> unordered_set<T, Hash, equal_to<T>, Allocator>;
}
```

```
template<class InputIterator>
unordered_set(InputIterator f, InputIterator l,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
template<container-compatible-range<value_type> R>
unordered_multiset(from_range_t, R&& rg, size_type n = see below,
    const hasher& hf = hasher(), const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
unordered_set(initializer_list<value_type> il,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
```

Effects: Constructs an empty `unordered_set` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f, l)` **for the first form, or from the range `[il.begin(), il.end())` for the second form, `rg`, or `il`, respectively.** `max_load_factor()` returns 1.0.

Complexity: Average case linear, worst case quadratic.

❖ Class template `unordered_multiset`

[unord.multiset]

❖ Overview

[unord.multiset.overview]

```
namespace std {
template<class Key,
class Hash = hash<Key>,
class Pred = equal_to<Key>,
class Allocator = allocator<Key>>
class unordered_multiset {
public:
    // types

    // ??, construct/copy/destroy
    unordered_multiset();
```

```

explicit unordered_multiset(size_type n,
const hasher& hf = hasher(),
const key_equal& eql = key_equal(),
const allocator_type& a = allocator_type());
template<class InputIterator>
unordered_multiset(InputIterator f, InputIterator l,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());

```

```

template<container-compatible-range<value\_type> R>
unordered\_multiset\(from\_range\_t, R&& rg, size\_type n = see below,
const hasher& hf = hasher\(\), const key\_equal& eql = key\_equal\(\),
const allocator\_type& a = allocator\_type\(\)\);

```

```

unordered_multiset(const unordered_multiset&);
unordered_multiset(unordered_multiset&&);
explicit unordered_multiset(const Allocator&);
unordered_multiset(const unordered_multiset&, const Allocator&);
unordered_multiset(unordered_multiset&&, const Allocator&);
unordered_multiset(initializer_list<value_type> il,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
unordered_multiset(size_type n, const allocator_type& a)
: unordered_multiset(n, hasher(), key_equal(), a) { }
unordered_multiset(size_type n, const hasher& hf, const allocator_type& a)
: unordered_multiset(n, hf, key_equal(), a) { }
template<class InputIterator>
unordered_multiset(InputIterator f, InputIterator l, size_type n, const allocator_type& a)
: unordered_multiset(f, l, n, hasher(), key_equal(), a) { }
template<class InputIterator>
unordered_multiset(InputIterator f, InputIterator l, size_type n, const hasher& hf,
const allocator_type& a)
: unordered_multiset(f, l, n, hf, key_equal(), a) { }

```

```

template<container-compatible-range<value\_type> R>
unordered\_multiset\(from\_range\_t, R&& rg, size\_type n, const allocator\_type& a\)
: unordered\_multiset\(from\_range, std::forward<R>\(rg\), n, hasher\(\), key\_equal\(\), a\) {}

```

```

template<container-compatible-range<value\_type> R>
unordered\_multiset\(from\_range\_t, R&& rg, size\_type n, const hasher& hf, const allocator\_type&
a)
: unordered\_multiset\(from\_range, std::forward<R>\(rg\), n, hf, key\_equal\(\), a\) {}

```

```

unordered_multiset(initializer_list<value_type> il, size_type n, const allocator_type& a)
: unordered_multiset(il, n, hasher(), key_equal(), a) { }
unordered_multiset(initializer_list<value_type> il, size_type n, const hasher& hf,
const allocator_type& a)

```



```

: unordered_multiset(il, n, hf, key_equal(), a) { }
~unordered_multiset();

iterator insert(const value_type& obj);
iterator insert(value_type&& obj);
iterator insert(const_iterator hint, const value_type& obj);
iterator insert(const_iterator hint, value_type&& obj);
template<class InputIterator> void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value\_type> R>
void insert\_range\(R&& rg\);
void insert(initializer_list<value_type>);

};

template<class InputIterator,
class Hash = hash<iter-value-type<InputIterator>>,
class Pred = equal_to<iter-value-type<InputIterator>>,
class Allocator = allocator<iter-value-type<InputIterator>>>
unordered_multiset(InputIterator, InputIterator, see below::size_type = see below,
Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multiset<iter-value-type<InputIterator>,
Hash, Pred, Allocator>;

template<ranges::input_range R,
class Hash = hash<ranges::range_value_t<R>>,
class Pred = equal_to<ranges::range_value_t<R>>,
class Allocator = allocator<ranges::range_value_t<R>>>
unordered_multiset(from_range_t, R&&, typename see below::size_type = see below,
Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multiset<ranges::range_value_t<R>, Hash, Pred, Allocator>;

template<class T, class Hash = hash<T>,
class Pred = equal_to<T>, class Allocator = allocator<T>>
unordered_multiset(initializer_list<T>, typename see below::size_type = see below,
Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multiset<T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_multiset(InputIterator, InputIterator, typename see below::size_type, Allocator)
-> unordered_multiset<iter-value-type<InputIterator>,
hash<iter-value-type<InputIterator>>,
equal_to<iter-value-type<InputIterator>>,
Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_multiset(InputIterator, InputIterator, typename see below::size_type,
Hash, Allocator)
-> unordered_multiset<iter-value-type<InputIterator>, Hash,
equal_to<iter-value-type<InputIterator>>,
Allocator>;

```

```

template<ranges::input_range R, class Allocator>
unordered_multiset(from_range_t, R&&, typename see below::size_type, Allocator)
-> unordered_multiset<ranges::range_value_t<R>,
    hash<ranges::range_value_t<R>>,
    equal_to<ranges::range_value_t<R>>, Allocator>;

template<ranges::input_range R, class Allocator>
unordered_multiset(from_range_t, R&&, Allocator)
-> unordered_multiset<ranges::range_value_t<R>,
    hash<ranges::range_value_t<R>>,
    equal_to<ranges::range_value_t<R>>, Allocator>;

template<ranges::input_range R, class Hash, class Allocator>
unordered_multiset(from_range_t, R&&, typename see below::size_type, Hash, Allocator)
-> unordered_multiset<ranges::range_value_t<R>,
    Hash,
    equal_to<ranges::range_value_t<R>>, Allocator>;

template<class T, class Allocator>
unordered_multiset(initializer_list<T>, typename see below::size_type, Allocator)
-> unordered_multiset<T, hash<T>, equal_to<T>, Allocator>;

template<class T, class Hash, class Allocator>
unordered_multiset(initializer_list<T>, typename see below::size_type, Hash, Allocator)
-> unordered_multiset<T, Hash, equal_to<T>, Allocator>;
}

```



Constructors

[unord.multiset.cnstr]

```

template<class InputIterator>
unordered_multiset(InputIterator f, InputIterator l,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
template<container-compatible-range<value_type> R>
unordered_multiset(from_range_t, R&& rg, size_type n = see below,
    const hasher& hf = hasher(), const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());
unordered_multiset(initializer_list<value_type> il,
    size_type n = see below,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());

```

Effects: Constructs an empty `unordered_multiset` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f, l)` **for the first form, or from the range `[il.begin(), il.end())` for the second form, rg, or il, respectively**. `max_load_factor()` returns 1.0.

Complexity: Average case linear, worst case quadratic.



Container adaptors

[container.adaptors]



Definition

[queue.defn]

```
namespace std {
    template<class T, class Container = deque<T>>
    class queue {
    public:
        using value_type      = typename Container::value_type;
        using reference        = typename Container::reference;
        using const_reference  = typename Container::const_reference;
        using size_type        = typename Container::size_type;
        using container_type    = Container;

    protected:
        Container c;

    public:
        queue() : queue(Container()) {}
        explicit queue(const Container&);
        explicit queue(Container&&);
        template<class InputIterator>
        queue(InputIterator first, InputIterator last);
        template<container-compatible-range<T> R>
        queue\(from\_range\_t, R&& rg\);

        template<class Alloc> explicit queue(const Alloc&);
        template<class Alloc> queue(const Container&, const Alloc&);
        template<class Alloc> queue(Container&&, const Alloc&);
        template<class Alloc> queue(const queue&, const Alloc&);
        template<class Alloc> queue(queue&&, const Alloc&);

        template<class InputIterator, class Alloc>
        queue(InputIterator first, InputIterator last, const Alloc&);
        template<container-compatible-range<T> R, class Alloc>
        queue\(from\_range\_t, R&& rg, const Alloc&\);

        [[nodiscard]] bool empty() const    { return c.empty(); }
        size_type size() const               { return c.size(); }
        reference front()                    { return c.front(); }
        const_reference front() const        { return c.front(); }
        reference back()                     { return c.back(); }
        const_reference back() const         { return c.back(); }
        void push(const value_type& x)       { c.push_back(x); }
        void push(value_type&& x)            { c.push_back(std::move(x)); }
        template<container-compatible-range<T> R>
        void push\_range\(R&& rg\);
    };
}
```

```

    template<class... Args>
    decltype(auto) emplace(Args&&... args)
    { return c.emplace_back(std::forward<Args>(args)...); }
    void pop() { c.pop_front(); }
    void swap(queue& q) noexcept(is_nothrow_swappable_v<Container>)
    { using std::swap; swap(c, q.c); }
};

template<class Container>
queue(Container) -> queue<typename Container::value_type, Container>;

template<class InputIterator>
queue(InputIterator, InputIterator) -> queue<iter-value-type<InputIterator>>;

template<ranges::input\_range R>
queue\(from\_range\_t, R&&\)
-> queue<ranges::range\_value\_t<R>>;

template<class Container, class Allocator>
queue(Container, Allocator) -> queue<typename Container::value_type, Container>;

template<class InputIterator, class Allocator>
queue(InputIterator, InputIterator, Allocator)
-> queue<iter-value-type<InputIterator>, deque<iter-value-type<InputIterator>, Allocator>>;

template<ranges::input\_range R, class Allocator>
queue\(from\_range\_t, R&&, Allocator\)
-> queue<ranges::range\_value\_t<R>, deque<ranges::range\_value\_t<R>, Allocator>>;

template<class T, class Container>
void swap(queue<T, Container>& x, queue<T, Container>& y) noexcept(noexcept(x.swap(y)));

template<class T, class Container, class Alloc>
struct uses_allocator<queue<T, Container>, Alloc>
: uses_allocator<Container, Alloc>::type { };
}

```



Constructors

[queue.cons]

```
explicit queue(const Container& cont);
```

Effects: Initializes c with cont.

```
explicit queue(Container&& cont);
```

Effects: Initializes c with std::move(cont).

```
template<class InputIterator>
queue(InputIterator first, InputIterator last);
```

Effects: Initializes c with first as the first argument and last as the second argument.

```
template<container-compatible-range<T> R>
queue(from_range_t, R&& rg);
```

Effects: Initializes `c` with `ranges::to<Container>(std::forward<R>(rg))`.

❖ Constructors with allocators

[queue.cons.alloc]

[...]

```
template<class Alloc> queue(const queue& q, const Alloc& a);
```

Effects: Initializes `c` with `q.c` as the first argument and `a` as the second argument.

```
template<class Alloc> queue(queue&& q, const Alloc& a);
```

Effects: Initializes `c` with `std::move(q.c)` as the first argument and `a` as the second argument.

```
template<class InputIterator, class Alloc>
queue(InputIterator first, InputIterator last, const Alloc & alloc);
```

Effects: Initializes `c` with `first` as the first argument, `last` as the second argument and `alloc` as the third argument.

```
template<container-compatible-range<T> R, class Alloc>
queue(from_range_t, R&& rg, const Alloc& a);
```

Effects: Initializes `c` with `ranges::to<Container>(std::forward<R>(rg), a)`.

...

```
template<container-compatible-range<T> R>
void push_range(R&& rg);
```

Effects: Equivalent to `c.append_range(std::forward<R>(rg))` if that is a valid expression, otherwise:

```
ranges::copy(rg, back_inserter(c));
```

❖ Class template `priority_queue`

[priority.queue]

❖ Overview

[priorityqueue.overview]

Any sequence container with random access iterator and supporting operations `front()`, `push_back()` and `pop_back()` can be used to instantiate `priority_queue`. In particular, `vector` and `deque` can be used. Instantiating `priority_queue` also involves supplying a function or function object for making priority comparisons; the library assumes that the function or function object defines a strict weak ordering.

```
namespace std {
template<class T, class Container = vector<T>,
```

```

class Compare = less<typename Container::value_type>>
class priority_queue {
public:
    using value_type      = typename Container::value_type;
    using reference       = typename Container::reference;
    using const_reference = typename Container::const_reference;
    using size_type       = typename Container::size_type;
    using container_type  = Container;
    using value_compare    = Compare;

protected:
    Container c;
    Compare comp;

public:
    priority_queue() : priority_queue(Compare()) {}
    explicit priority_queue(const Compare& x) : priority_queue(x, Container()) {}
    priority_queue(const Compare& x, const Container&);
    priority_queue(const Compare& x, Container&&);
    template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last, const Compare& x,
        const Container&);
    template<class InputIterator>
    priority_queue(InputIterator first, InputIterator last,
        const Compare& x = Compare(), Container&& = Container());

    template<container-compatible-range<T> R>
    priority\_queue\(from\_range\_t, R&& rg, const Compare& x = Compare\(\)\);

    template<class Alloc> explicit priority_queue(const Alloc&);
    template<class Alloc> priority_queue(const Compare&, const Alloc&);
    template<class Alloc> priority_queue(const Compare&, const Container&, const Alloc&);
    template<class Alloc> priority_queue(const Compare&, Container&&, const Alloc&);
    template<class Alloc> priority_queue(const priority_queue&, const Alloc&);
    template<class Alloc> priority_queue(priority_queue&&, const Alloc&);

    template<container-compatible-range<T> R, class Alloc>
    priority\_queue\(from\_range\_t, R&& rg, const Compare&, const Alloc&\);
    template<container-compatible-range<T> R, class Alloc>
    priority\_queue\(from\_range\_t, R&& rg, const Alloc&\);

    [[nodiscard]] bool empty() const { return c.empty(); }
    size_type size() const { return c.size(); }
    const_reference top() const { return c.front(); }
    void push(const value_type& x);
    void push(value_type&& x);
    template<container-compatible-range<T> R>
    void push\_range\(R&& rg\);
    template<class... Args> void emplace(Args&&... args);

```

```

    void pop();
    void swap(priority_queue& q) noexcept(is_nothrow_swappable_v<Container> &&
        is_nothrow_swappable_v<Compare>)
    { using std::swap; swap(c, q.c); swap(comp, q.comp); }
};

template<class Compare, class Container>
priority_queue(Compare, Container)
-> priority_queue<typename Container::value_type, Container, Compare>;

template<class InputIterator,
class Compare = less<typename iterator_traits<InputIterator>::value_type>,
class Container = vector<typename iterator_traits<InputIterator>::value_type>>
priority_queue(InputIterator, InputIterator, Compare = Compare(), Container = Container())
-> priority_queue<typename iterator_traits<InputIterator>::value_type, Container, Compare>;

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>
priority_queue(from_range_t, R&&, Compare = Compare())
-> priority_queue<ranges::range_value_t<R>, vector<ranges::range_value_t<R>>, Compare>;

template<class Compare, class Container, class Allocator>
priority_queue(Compare, Container, Allocator)
-> priority_queue<typename Container::value_type, Container, Compare>;

template<ranges::input_range R,
class Compare,
class Allocator>
priority_queue(from_range_t, R&&, Compare, Allocator)
-> priority_queue<ranges::range_value_t<R>,
vector<ranges::range_value_t<R>, Allocator>, Compare>;

template<ranges::input_range R, class Allocator>
priority_queue(from_range_t, R&&, Allocator)
-> priority_queue<ranges::range_value_t<R>,
vector<ranges::range_value_t<R>, Allocator>>;

// no equality is provided

template<class T, class Container, class Compare, class Alloc>
struct uses_allocator<priority_queue<T, Container, Compare>, Alloc>
: uses_allocator<Container, Alloc>::type { };
}

```



Constructors

[priority_queue.cons]

```

priority_queue(const Compare& x, const Container& y);
priority_queue(const Compare& x, Container&& y);

```

Preconditions: x defines a strict weak ordering.

Effects: Initializes comp with x and c with y (copy constructing or move constructing as

appropriate); calls `make_heap(c.begin(), c.end(), comp)`.

```
template<class InputIterator>
priority_queue(InputIterator first, InputIterator last, const Compare& x, const Container& y);
template<class InputIterator>
priority_queue(InputIterator first, InputIterator last, const Compare& x = Compare(),
Container&& y = Container());
```

Preconditions: `x` defines a strict weak ordering.

Effects: Initializes `comp` with `x` and `c` with `y` (copy constructing or move constructing as appropriate); calls `c.insert(c.end(), first, last)`; and finally calls `make_heap(c.begin(), c.end(), comp)`.

```
template<container-compatible-range<T> R>
priority_queue(from_range_t, R&& rg, const Compare& x = Compare());
```

Preconditions: `x` defines a strict weak ordering.

Effects: Initializes `comp` with `x` and `c` with `ranges::to<Container>(std::forward<R>(rg))` and finally calls `make_heap(c.begin(), c.end(), comp)`.



Constructors with allocators

[priority_queue.cons.alloc]

If `uses_allocator_v<container_type, Alloc>` is false the constructors in this subclause shall not participate in overload resolution.

```
template<class Alloc> explicit priority_queue(const Alloc& a);
```

Effects: Initializes `c` with `a` and value-initializes `comp`.

```
template<class Alloc> priority_queue(const Compare& compare, const Alloc& a);
```

Effects: Initializes `c` with `a` and initializes `comp` with `compare`.

```
template<class Alloc>
priority_queue(const Compare& compare, const Container& cont, const Alloc& a);
```

Effects: Initializes `c` with `cont` as the first argument and `a` as the second argument, and initializes `comp` with `compare`; calls `make_heap(c.begin(), c.end(), comp)`.

```
template<class Alloc>
priority_queue(const Compare& compare, Container&& cont, const Alloc& a);
```

Effects: Initializes `c` with `std::move(cont)` as the first argument and `a` as the second argument, and initializes `comp` with `compare`; calls `make_heap(c.begin(), c.end(), comp)`.

```
template<class Alloc> priority_queue(const priority_queue& q, const Alloc& a);
```

Effects: Initializes `c` with `q.c` as the first argument and `a` as the second argument, and initializes `comp` with `q.comp`.

```
template<class Alloc> priority_queue(priority_queue&& q, const Alloc& a);
```


Effects: Initializes `c` with `std::move(q.c)` as the first argument and `a` as the second argument, and initializes `comp` with `std::move(q.comp)`.

```
template<container-compatible-range<T> R, class Alloc>
priority_queue(from_range_t, R&& rg, const Compare& compare, const Alloc& a);
```

Effects: Initializes `comp` with `compare` and `c` with `ranges::to<Container>(std::forward<R>(rg), a)`; calls `make_heap(c.begin(), c.end(), comp)`.

```
template<container-compatible-range<T> R, class Alloc>
priority_queue(from_range_t, R&& rg, const Alloc& a);
```

Effects: Initializes `c` with `ranges::to<Container>(std::forward<R>(rg), a)`; calls `make_heap(c.begin(), c.end(), comp)`.

◆ Members

[priqueue.members]

```
void push(const value_type& x);
```

Effects: As if by:

```
c.push_back(x);
push_heap(c.begin(), c.end(), comp);
```

```
void push(value_type&& x);
```

Effects: As if by:

```
c.push_back(std::move(x));
push_heap(c.begin(), c.end(), comp);
```

```
template<container-compatible-range<T> R>
void push_range(R&& rg);
```

Effects: Insert all elements of `rg` in `c`.

Postconditions: `is_heap(c.begin(), c.end(), comp)` is true.

```
template<class... Args> void emplace(Args&&... args);
```

Effects: As if by:

```
c.emplace_back(std::forward<Args>(args)...);
push_heap(c.begin(), c.end(), comp);
```

◆ Class template stack

[stack]

◆ Definition

[stack.defn]

```

namespace std {
    template<class T, class Container = deque<T>>
    class stack {
    public:
        using value_type      = typename Container::value_type;
        using reference        = typename Container::reference;
        using const_reference  = typename Container::const_reference;
        using size_type        = typename Container::size_type;
        using container_type   = Container;

    protected:
        Container c;

    public:
        stack() : stack(Container()) {}
        explicit stack(const Container&);
        explicit stack(Container&&);

        template<class InputIterator>
        stack(InputIterator first, InputIterator last);
        template<container-compatible-range<T> R>
        stack\(from\_range\_t, R&& rg\);

        template<class Alloc> explicit stack(const Alloc&);
        template<class Alloc> stack(const Container&, const Alloc&);
        template<class Alloc> stack(Container&&, const Alloc&);
        template<class Alloc> stack(const stack&, const Alloc&);
        template<class Alloc> stack(stack&&, const Alloc&);

        template<class InputIterator, class Alloc>
        stack(InputIterator first, InputIterator last, const Alloc&);
        stack<container-compatible-range<T> R, class Alloc>
        stack\(from\_range\_t, R&& rg, const Alloc&\);

        [[nodiscard]] bool empty() const    { return c.empty(); }
        size_type size() const               { return c.size(); }
        reference top()                     { return c.back(); }
        const_reference top() const          { return c.back(); }
        void push(const value_type& x)       { c.push_back(x); }
        void push(value_type&& x)            { c.push_back(std::move(x)); }
        template<container-compatible-range<T> R>
        void push\_range\(R&& rg\);
        template<class... Args>
        decltype(auto) emplace(Args&&... args)
        { return c.emplace_back(std::forward<Args>(args)...); }
        void pop()                          { c.pop_back(); }
        void swap(stack& s) noexcept(is_nothrow_swappable_v<Container>)
        { using std::swap; swap(c, s.c); }
    };

```

```

template<class Container>
stack(Container) -> stack<typename Container::value_type, Container>;

template<class InputIterator>
stack(InputIterator, InputIterator)
-> stack<iter-value-type<InputIterator>>;

template<ranges::input_range R>
stack(from_range_t, R&&)
-> stack<ranges::range_value_t<R>>;

template<class Container, class Allocator>
stack(Container, Allocator) -> stack<typename Container::value_type, Container>;

template<class InputIterator, class Allocator>
stack(InputIterator, InputIterator, Allocator)
-> stack<iter-value-type<InputIterator>,
stack<iter-value-type<InputIterator>, Allocator>>;

template<ranges::input_range R, class Allocator>
stack(from_range_t, R&&, Allocator)
-> stack<ranges::range_value_t<R>, deque<ranges::range_value_t<R>, Allocator>>;

template<class T, class Container, class Alloc>
struct uses_allocator<stack<T, Container>, Alloc>
: uses_allocator<Container, Alloc>::type { };
}

```



Constructors

[stack.cons]

```
explicit stack(const Container& cont);
```

Effects: Initializes c with cont.

```
explicit stack(Container&& cont);
```

Effects: Initializes c with std::move(cont).

```
template<class InputIterator>
stack(InputIterator first, InputIterator last);
```

Effects: Initializes c with first as the first argument and last as the second argument.

```
template<container-compatible-range<T> R>
stack(from_range_t, R&& rg);
```

Effects: Initializes c with ranges::to<Container>(std::forward<R>(rg)).



Constructors with allocators

[stack.cons.alloc]

[...]

```
template<class Alloc> stack(const stack& s, const Alloc& a);
```

Effects: Initializes `c` with `s.c` as the first argument and `a` as the second argument.

```
template<class Alloc> stack(stack&& s, const Alloc& a);
```

Effects: Initializes `c` with `std::move(s.c)` as the first argument and `a` as the second argument.

```
template<class InputIterator, class Alloc>
stack(InputIterator first, InputIterator last, const Alloc& alloc);
```

Effects: Initializes `c` with `first` as the first argument, `last` as the second argument and `alloc` as the third argument.

```
template<container-compatible-range<T> R, class Alloc>
stack(from_range_t, R&& rg, const Alloc& a);
```

Effects: Initializes `c` with `ranges::to<Container>(std::forward<R>(rg), a)`.

```
template<container-compatible-range<T> R>
void push_range(R&& rg);
```

Effects: Equivalent to `c.append_range(std::forward<R>(rg))` if that is a valid expression, otherwise:

```
ranges::copy(rg, back_inserter(c));
```



Strings library

[strings]



Class template `basic_string`

[basic.string]

```
namespace std {
    template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
    class basic_string {
    public:

        // ??, construct/copy/destroy
        constexpr basic_string() noexcept(noexcept(Allocator())) : basic_string(Allocator()) { }
        constexpr explicit basic_string(const Allocator& a) noexcept;
        constexpr basic_string(const basic_string& str);
        constexpr basic_string(basic_string&& str) noexcept;
        constexpr basic_string(const basic_string& str, size_type pos,
            const Allocator& a = Allocator());
        constexpr basic_string(const basic_string& str, size_type pos, size_type n,
            const Allocator& a = Allocator());
        template<class T>
        constexpr basic_string(const T& t, size_type pos, size_type n,
            const Allocator& a = Allocator());
        template<class T>
```

```

constexpr explicit basic_string(const T& t, const Allocator& a = Allocator());
constexpr basic_string(const charT* s, size_type n, const Allocator& a = Allocator());
constexpr basic_string(const charT* s, const Allocator& a = Allocator());
constexpr basic_string(size_type n, charT c, const Allocator& a = Allocator());
template<class InputIterator>
constexpr basic_string(InputIterator begin, InputIterator end, const Allocator& a = Allocator());

template<container-compatible-range<charT> R>
constexpr basic\_string\(from\_range\_t, R&& rg, const Allocator& a = Allocator\(\)\);

constexpr basic_string(initializer_list<charT>, const Allocator& = Allocator());
constexpr basic_string(const basic_string&, const Allocator&);
constexpr basic_string(basic_string&&, const Allocator&);
constexpr ~basic_string();

// ??, modifiers
constexpr basic_string& operator+=(const basic_string& str);
template<class T>
constexpr basic_string& operator+=(const T& t);
constexpr basic_string& operator+=(const charT* s);
constexpr basic_string& operator+=(charT c);
constexpr basic_string& operator+=(initializer_list<charT>);
constexpr basic_string& append(const basic_string& str);
constexpr basic_string& append(const basic_string& str, size_type pos, size_type n = npos);
template<class T>
constexpr basic_string& append(const T& t);
template<class T>
constexpr basic_string& append(const T& t, size_type pos, size_type n = npos);
constexpr basic_string& append(const charT* s, size_type n);
constexpr basic_string& append(const charT* s);
constexpr basic_string& append(size_type n, charT c);
template<class InputIterator>
constexpr basic_string& append(InputIterator first, InputIterator last);
constexpr basic_string& append(initializer_list<charT>);

constexpr void push_back(charT c);
template<container-compatible-range<charT> R>
constexpr basic\_string& append\_range\(R&& rg\);

constexpr basic_string& assign(const basic_string& str);
constexpr basic_string& assign(basic_string&& str)
noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
allocator_traits<Allocator>::is_always_equal::value);
constexpr basic_string& assign(const basic_string& str, size_type pos, size_type n = npos);
template<class T>
constexpr basic_string& assign(const T& t);
template<class T>
constexpr basic_string& assign(const T& t, size_type pos, size_type n = npos);
constexpr basic_string& assign(const charT* s, size_type n);
constexpr basic_string& assign(const charT* s);
constexpr basic_string& assign(size_type n, charT c);

```

```

template<class InputIterator>
constexpr basic_string& assign(InputIterator first, InputIterator last);

template<container-compatible-range<charT> R>
constexpr basic\_string& assign\_range\(R&& rg\);

constexpr basic_string& assign(initializer_list<charT>);

constexpr basic_string& insert(size_type pos, const basic_string& str);
constexpr basic_string& insert(size_type pos1, const basic_string& str,
size_type pos2, size_type n = npos);
template<class T>
constexpr basic_string& insert(size_type pos, const T& t);
template<class T>
constexpr basic_string& insert(size_type pos1, const T& t,
size_type pos2, size_type n = npos);
constexpr basic_string& insert(size_type pos, const charT* s, size_type n);
constexpr basic_string& insert(size_type pos, const charT* s);
constexpr basic_string& insert(size_type pos, size_type n, charT c);
constexpr iterator insert(const_iterator p, charT c);
constexpr iterator insert(const_iterator p, size_type n, charT c);
template<class InputIterator>
constexpr iterator insert(const_iterator p, InputIterator first, InputIterator last);

template<container-compatible-range<charT> R>
constexpr iterator insert\_range\(const\_iterator p, R&& rg\);

constexpr iterator insert(const_iterator p, initializer_list<charT>);

//...

constexpr basic_string& replace(size_type pos1, size_type n1, const basic_string& str);
constexpr basic_string& replace(size_type pos1, size_type n1, const basic_string& str,
size_type pos2, size_type n2 = npos);
template<class T>
constexpr basic_string& replace(size_type pos1, size_type n1, const T& t);
template<class T>
constexpr basic_string& replace(size_type pos1, size_type n1, const T& t,
size_type pos2, size_type n2 = npos);
constexpr basic_string& replace(size_type pos, size_type n1, const charT* s, size_type n2);
constexpr basic_string& replace(size_type pos, size_type n1, const charT* s);
constexpr basic_string& replace(size_type pos, size_type n1, size_type n2, charT c);
constexpr basic_string& replace(const_iterator i1, const_iterator i2,
const basic_string& str);
template<class T>
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const T& t);
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const charT* s,
size_type n);
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const charT* s);
constexpr basic_string& replace(const_iterator i1, const_iterator i2, size_type n, charT c);
template<class InputIterator>

```

```

constexpr basic_string& replace(const_iterator i1, const_iterator i2,
    InputIterator j1, InputIterator j2);

template<container-compatible-range<charT> R>
constexpr basic_string& replace_with_range(const_iterator i1, const_iterator i2, R&& rg);

constexpr basic_string& replace(const_iterator, const_iterator, initializer_list<charT>);

//....
};

template<class InputIterator,
class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
basic_string(InputIterator, InputIterator, Allocator = Allocator())
-> basic_string<typename iterator_traits<InputIterator>::value_type,
char_traits<typename iterator_traits<InputIterator>::value_type>,
Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
basic_string(from_range_t, R&&, Allocator = Allocator())
-> basic_string<ranges::range_value_t<R>, char_traits<ranges::range_value_t<R>>, Allocator>;

template<class charT,
class traits,
class Allocator = allocator<charT>>
explicit basic_string(basic_string_view<charT, traits>, const Allocator& = Allocator())
-> basic_string<charT, traits, Allocator>;

template<class charT,
class traits,
class Allocator = allocator<charT>>
basic_string(basic_string_view<charT, traits>,
typename see below::size_type, typename see below::size_type,
const Allocator& = Allocator())
-> basic_string<charT, traits, Allocator>;
}

```

◆ Constructors and assignment operators

[string.cons]

```

template<class InputIterator>
constexpr basic_string(InputIterator begin, InputIterator end, const Allocator& a = Allocator());

```

Constraints: InputIterator is a type that qualifies as an input iterator.

Effects: Constructs a string from the values in the range [begin, end), as indicated in [container.seq.req].

```

template<container-compatible-range<charT> R>
constexpr basic_string(from_range_t, R&& rg, const Allocator& = Allocator());

```

Effects: Constructs a string from the values in the range rg as indicated in [container.seq.req].

❖ **Modifiers** [string.modifiers]

❖ **basic_string::operator+=** [string.op.append]

```
template<class InputIterator>
constexpr basic_string& append(InputIterator first, InputIterator last);
```

Constraints: InputIterator is a type that qualifies as an input iterator.

Effects: Equivalent to: return append(basic_string(first, last, get_allocator()));

```
template<container-compatible-range<charT> R>
constexpr basic_string& append_range(R&& rg);
```

Effects: Equivalent to: return append(basic_string(from_range, std::forward<R>(rg), get_allocator()));

```
constexpr void push_back(charT c);
```

Effects: Equivalent to append(size_type{1}, c).

❖ **basic_string::assign** [string.assign]

```
template<class InputIterator>
constexpr basic_string& assign(InputIterator first, InputIterator last);
```

Constraints: InputIterator is a type that qualifies as an input iterator.

Effects: Equivalent to: return assign(basic_string(first, last, get_allocator()));

```
template<container-compatible-range<charT> R>
constexpr basic_string& assign_range(R&& rg);
```

Effects: Equivalent to: return assign(basic_string(from_range, std::forward<R>(rg), get_allocator()));

❖ **basic_string::insert** [string.insert]

```
template<class InputIterator>
constexpr iterator insert(const_iterator p, InputIterator first, InputIterator last);
```

Constraints: InputIterator is a type that qualifies as an input iterator.

Preconditions: p is a valid iterator on *this.

Effects: Equivalent to insert(p - begin(), basic_string(first, last, get_allocator())).

Returns: An iterator which refers to the first inserted character, or p if first == last.

```
template<container-compatible-range<charT> R>
constexpr iterator insert_range(const_iterator p, R&& rg);
```


Preconditions: *p* is a valid iterator on **this*.

Effects: Equivalent to `insert(p - begin(), basic_string(from_range, std::forward<R>(rg), get_allocator()))`.

Returns: An iterator which refers to the first inserted character, or *p* if *rg* is empty.

```
constexpr iterator insert(const_iterator p, initializer_list<charT> il);
```

Effects: Equivalent to: `return insert(p, il.begin(), il.end());`



basic_string::replace

[string.replace]

```
template<class InputIterator>
constexpr basic_string& replace(const_iterator i1, const_iterator i2,
InputIterator j1, InputIterator j2);
```

Constraints: `InputIterator` is a type that qualifies as an input iterator.

Effects: Equivalent to: `return replace(i1, i2, basic_string(j1, j2, get_allocator()));`

```
template<container-compatible-range<charT> R>
constexpr basic_string& replace_with_range(const_iterator i1, const_iterator i2, R&& rg);
```

Effects: Equivalent to: `return replace(i1, i2, basic_string(from_range, std::forward<R>(rg), get_allocator()));`

```
constexpr basic_string& replace(const_iterator i1, const_iterator i2, initializer_list<charT> il);
```

Effects: Equivalent to: `return replace(i1, i2, il.begin(), il.size());`



Header <ranges> synopsis

[ranges.syn]

```
#include <compare>           // see ??
#include <initializer_list>    // see ??
#include <iterator>           // see ??
```

```
namespace std::ranges {
    template<class R>
    using keys_view = elements_view<views::all_t<R>, 0>;
    template<class R>
    using values_view = elements_view<views::all_t<R>, 1>;
```

```
namespace views {
    template<size_t N>
    inline constexpr unspecified elements = unspecified ;
    inline constexpr auto keys = elements<0>;
    inline constexpr auto values = elements<1>;
}
```

```
template<class C, input_range R, class... Args>
```

```

    requires (!view<C>)
    constexpr C to(R&& r, Args&&... args);

    template<template<class...> class C, input_range R, class... Args>
    constexpr auto to(R && r, Args&&... args) -> see below;

    template<class C, class... Args>
    requires (!view<C>)
    constexpr auto to(Args&&... args) -> see below;

    template<template<class...> class C, class... Args>
    constexpr auto to(Args&&... args) -> see below;
}
namespace std {
    namespace views = ranges::views;

    template<class I, class S, ranges::subrange_kind K>
    struct tuple_size<ranges::subrange<I, S, K>>
    : integral_constant<size_t, 2> {};
    template<class I, class S, ranges::subrange_kind K>
    struct tuple_element<0, ranges::subrange<I, S, K>> {
        using type = I;
    };
    template<class I, class S, ranges::subrange_kind K>
    struct tuple_element<1, ranges::subrange<I, S, K>> {
        using type = S;
    };
    template<class I, class S, ranges::subrange_kind K>
    struct tuple_element<0, const ranges::subrange<I, S, K>> {
        using type = I;
    };
    template<class I, class S, ranges::subrange_kind K>
    struct tuple_element<1, const ranges::subrange<I, S, K>> {
        using type = S;
    };

    struct from_range_t { explicit from_range_t() = default; };
    inline constexpr from_range_t from_range{};
}

```

In [range.utility] Insert after section [range.dangling]



Range conversions

[range.utility.conversions]

The range conversion functions construct an object (usually a container) from a range, by using a constructor taking a range, a `from_range_t` tagged constructor, or a constructor taking a pair of iterators, or by inserting each element of the range into the default-constructed object.

`ranges::to` is applied recursively, allowing the conversion of a range of ranges.

[Example:

```
string_view str = "the quick brown fox";
auto words = views::split(str, ' ') | to<vector<string>>();
// returns vector<string>{"the", "quick", "brown", "fox"}
```

— *end example*]

Let *reservable-container* be defined as follows:

```
template<class Container>
constexpr bool reservable-container = // exposition only
    sized_range<Container> &&
    requires(Container& c, range_size_t<Container> n) {
        c.reserve(n);
        {c.capacity()} -> same_as<decltype(n)>;
        {c.max_size()} -> same_as<decltype(n)>;
    };
```

Let *container-insertable* be defined as follows:

```
template<class Container, class Ref>
constexpr bool container-insertable = // exposition only
    requires(Container& c, Ref&& ref) {
        requires (requires { c.push_back(std::forward<Ref>(ref)); } ||
            requires { c.insert(c.end(), std::forward<Ref>(ref)); });
    };
```

Let *container-inserter* be defined as follows:

```
template <typename Ref>
auto container-inserter(C & c){ // exposition only
    if constexpr (requires { c.push_back(declval<Ref>()); })
        return back_inserter(c);
    else
        return inserter(c, c.end());
};
```



ranges::to

[range.utility.conversions.to]

```
template<class C, input_range R, class... Args>
requires (!view<C>)
constexpr C to(R&& r, Args&&... args);
```

Returns: An object of type C constructed from the elements of r in the following manner:

- If `convertible_to<range_reference_t<R>, range_value_t<C>>` is true:
 - If `constructible_from<C, R, Args...>` is true:


```
C(std::forward<R>(r), std::forward<Args>(args)...)

```
 - Otherwise if `constructible_from<C, from_range_t, R, Args...>` is true:


```
C(from_range, std::forward<R>(r), std::forward<Args>(args)...)

```

– Otherwise if

- * `common_range<R>` is true,
- * `cpp17-input-iterator<iterator_t<R>>` is true, and
- * `constructible_from<C, iterator_t<R>, sentinel_t<R>, Args...>` is true:
`C c(ranges::begin(r), ranges::end(r), std::forward<Args...>(args)...);`

– Otherwise if

- * `constructible_from<C, Args...>` is true, and
- * `container-insertable<C, range_reference_t<R>>` is true:
`C c(std::forward<Args...>(args)...);`
`if constexpr (sized_range<R> && reservable-container<C>)`
`c.reserve(ranges::size(r));`
`ranges::copy(r, container-inserter<range_reference_t<R>>(c));`

• Otherwise if `input_range<range_reference_t<R>>` is true:

```
to<C>(r | views::transform([](auto &&elem) {  
    return to<range_value_t<C>>(std::forward<decltype(elem)>(elem));  
})), std::forward<Args>(args)...);
```

• Otherwise the program is ill-formed.

```
template<template<class...> class C, input_range R, class... Args>  
constexpr auto to(R&& r, Args&&... args);
```

Let *input-iterator* be an exposition-only type

```
struct input-iterator { //exposition only  
    using iterator_category = input_iterator_tag;  
    using value_type = range_value_t<R>;  
    using difference_type = ptrdiff_t;  
    using pointer = add_pointer_t<range_reference_t<R>>;  
    using reference = range_reference_t<R>;  
    reference operator*() const;  
    pointer operator->() const;  
    input-iterator& operator++();  
    input-iterator operator++(int);  
    bool operator==(const input-iterator&) const;  
};
```

[*Note:* *input-iterator* meets the syntactic requirements of `Cpp17InputIterator`. — *end note*]

Let *DEDUCE_EXPR* be defined as follows:

- `C(declval<R>(), declval<Args>()...)` if that is a valid expression,
- otherwise, `C(from_range, declval<R>(), declval<Args>()...)` if that is a valid expression,

- otherwise, `C(declval<input-iterator>(), declval<input-iterator>(), declval<Args>()...)` if that is a valid expression,
- otherwise, the program is ill-formed.

Returns: `to<decltype(DEDUCE_EXPR)>(std::forward<R>(r), std::forward<Args>(args)...).`



ranges::to adaptors

[range.utility.conversions.adapters]

```
template<class C, class... Args>
requires (!view<C>)
constexpr auto to(Args&&... args);
```

```
template<template<class...> class C, class... Args>
constexpr auto to(Args&&... args);
```

Returns: A range adaptor closure object ([range.adaptor.object]) `f` that is a perfect forwarding call wrapper ([func.require]) with the following properties:

- It has no target object.
- Its bound argument entities `bound_args` consist of objects of types `decay_t<Args>...` direct-non-list-initialized with `std::forward<Args>(args)...`, respectively.
- Its call pattern is `to<C>(r, bound_args...)`, where `r` is the argument used in a function call expression of `f`.

Feature test macros

Add a new macro in `<version> __cpp_lib_ranges_to_container` set to the date of adoption. The macro `__cpp_lib_ranges_to_container` is also present in `<ranges>`.

Add a new macro in `<version> __cpp_lib_containers_ranges` set to the date of adoption. The macro `__cpp_lib_containers_ranges` is also present in `<vector>`, `<list>`, `<forward_list>` `<map>`, `<set>` `<unordered_map>`, `<unordered_set>`, `<deque>`, `<queue>`, `<priority_queue>`, `<stack>`, and `<string>`.

Implementation Experience

Implementations of ‘`ranges::to`’ are available in [RangeV3], [cmcstl2] and on Github [rangesnext]. The tagged ranges constructors, insert methods and other range-taking container members functions have **not** been implemented.

Related Paper and future work

Future work is needed to allow constructing `std::array` from *tiny-ranges*.

Acknowledgements

We would like to thank the people who gave feedback on this paper, notably Christopher Di Bella, Tomasz Kamiński, Barry Revzin and Tristan Brindle.

References

- [cmcstl2] <https://github.com/CaseyCarter/cmcstl2/blob/a7a714a9159b08adeb00a193e77b782846b3b20e/include/stl2/detail/to.hpp>
- [RangeV3] Eric Niebler https://github.com/ericniebler/range-v3/blob/v1.0-beta/include/range/v3/to_container.hpp
- [rangesnext] Corentin Jabot <https://github.com/cor3ntin/rangesnext/blob/master/include/cor3ntin/rangesnext/to.hpp>
- [CTAD Ranges] Eric Niebler <https://github.com/ericniebler/range-v3/blob/d284e9c84ff69bb416d9d94d029729dfb38c3364/include/range/v3/range/conversion.hpp#L140-L152>
- [P1391] Corentin Jabot *Range constructor for std::string_view*
<https://wg21.link/P1391>
- [N4885] Thomas Köppe *Working Draft, Standard for Programming Language C++*
<https://wg21.link/N4885>